

Received 27 February 2026, accepted 16 March 2026, date of publication 19 March 2026, date of current version 24 March 2026.

Digital Object Identifier 10.1109/ACCESS.2026.3675540

RESEARCH ARTICLE

Explainable Retail Demand Forecasting: A Hybrid LightGBM-SHAP Framework for Inventory Optimization

LALIT NARAYAN MISHRA¹, (Senior Member, IEEE),

BISWARANJAN SENAPATI², (Senior Member, IEEE),

SAROJ KUMAR NAYAK³, (Member, IEEE),

AND AMIT J. RANGARI⁴, (Senior Member, IEEE)

¹Lowe's Companies Inc., Mooresville, NC 28117, USA

²Department of Computer Science, UALR, Little Rock, AR 72204, USA

³Cognizant Technology Solutions, Concord, NC 28025, USA

⁴JPMorgan Chase, Atlanta, GA 30326, USA

Corresponding author: Lalit Narayan Mishra (Lnm8910@gmail.com)

ABSTRACT Gradient boosting models predict retail demand effectively, yet inventory planners distrust predictions they cannot interpret. This paper addresses the trust gap through a four-level Explainable AI (XAI) framework that pairs LightGBM prediction with layered interpretability: Permutation Feature Importance (PFI) for global ranking, Partial Dependence Plots (PDP) for marginal effects, SHapley Additive exPlanations (SHAP) for instance-level attribution, and Individual Conditional Expectation (ICE) plots for heterogeneity detection. ICE analysis, the key extension beyond prior work, reveals when averaged trends mask divergent product behavior, justifying segment-specific inventory policies. Validated on the UCI Online Retail II dataset with 41 engineered features, the proposed model achieves RMSE of 33.71 and R^2 of 0.243 on inherently intermittent stock-keeping unit (SKU)-level daily demand, significantly outperforming most baselines (paired t-test, $p < 0.001$; CatBoost not significant, $p = 0.917$). Ablation experiments confirm rolling statistics as the dominant feature group ($\Delta R^2 = -0.064$ upon removal). ICE heterogeneity analysis identifies product clusters where temporal features produce opposite effects, demonstrating that uniform reorder policies systematically misallocate inventory for specific segments.

INDEX TERMS Demand forecasting, explainable AI, LightGBM, SHAP, ICE plots, heterogeneity detection, retail analytics, inventory optimization, feature importance, gradient boosting.

I. INTRODUCTION

Retail inventory management operates on thin margins where carrying costs consume 20–30% of inventory value annually [1]. Gradient boosting algorithms, particularly LightGBM, have emerged as preferred solutions for complex demand patterns [2], [3], yet adoption stalls when planners cannot validate, debug, or explain predictions to procurement committees [4], [5], [6]. This trust deficit systematically degrades accuracy gains: planners override predictions they

do not understand, even when the model outperforms their heuristics [4].

Explainable AI (XAI) techniques offer a path forward [7]. Wang et al. [8] pioneered combining LightGBM with Permutation Feature Importance (PFI) and Partial Dependence Plots (PDP) for e-commerce prediction. However, their framework stops at global explanations, leaving two operational gaps. First, planners need instance-level attribution: which features pushed a specific anomalous forecast? Second, PDP averaging obscures heterogeneity that matters for inventory policy. If weekday effects differ between product segments, a single PDP curve misleads. Individual Conditional Expectation

The associate editor coordinating the review of this manuscript and approving it for publication was Jolanta Mizera-Pietraszko^{id}.

(ICE) plots reveal these divergences by tracing each product's response separately; crossing curves signal that uniform policies will misallocate inventory.

We address these gaps through a four-level XAI framework that extends Wang et al.'s approach with SHAP instance attribution and ICE heterogeneity detection. The research problem requires multi-level explanations: global methods for strategic planning (which features matter?), local methods for operational decisions (why this forecast?), and heterogeneity detection for segment differentiation (which products need separate treatment?).

A. RESEARCH CONTRIBUTIONS

We contribute four advances to explainable demand forecasting:

- 1) **Four-Level XAI Pipeline with Heterogeneity Detection:** We layer PFI (global ranking), PDP (marginal effects), SHAP (instance attribution), and ICE (individual response curves). The fourth level, ICE analysis, is the key extension beyond Wang et al. [8], exposing when averaged PDP curves mask divergent product behavior.
- 2) **Documented Feature Engineering Pipeline:** We engineer 41 features across five categories with published construction algorithms. Ablation experiments isolate each category's marginal contribution.
- 3) **Statistically Validated Evaluation:** We report paired t-tests, bootstrap confidence intervals, and Time-SeriesSplit cross-validation on the UCI Online Retail II dataset (1.07 million transactions).
- 4) **Policy-Oriented Interpretation:** Each XAI level maps to specific business decisions, from data pipeline monitoring (PFI) to segment-specific safety stock rules (ICE heterogeneity).

The remainder of this paper reviews related work (Section II), presents methodology (Section III), describes implementation (Section IV), reports results (Section V), discusses implications (Section VI), and concludes (Section VII).

II. RELATED WORK

We situate our contribution at the intersection of retail demand forecasting, gradient boosting for tabular data, and explainable AI. Our review emphasizes the methodological gap (absent local and heterogeneity-level explanations) that motivated the four-level XAI pipeline.

A. MACHINE LEARNING FOR RETAIL DEMAND FORECASTING

Classical methods, including Autoregressive Integrated Moving Average (ARIMA), exponential smoothing, and Holt-Winters [9], [10], dominated retail forecasting through the 2000s because their parameters translate into business language, but they assume linear, stationary relationships that promotional spikes and supply disruptions routinely violate [11]. Machine learning addressed these limitations: Random Forests [12] handle mixed feature types and missing

values, Support Vector Regression [13] separates non-linear patterns in high-dimensional spaces, and Long Short-Term Memory (LSTM) networks [14] model temporal dependencies, though long-range learning remains challenging [15].

Deep learning attracted attention through LSTMs [14], Temporal Fusion Transformers [16], and self-attention architectures [17]. Yet in the M5 Forecasting Competition, gradient boosting consistently outperformed deep learning on Walmart's 30,490-series dataset [3], likely because hand-crafted tabular features already encode relevant temporal patterns. Hybrid approaches like Prophet [18] and N-BEATS [19] seek compromise positions between accuracy and interpretability. None of these methods provides the transparent explanation that operations teams require for adoption.

B. GRADIENT BOOSTING METHODS

Gradient boosting builds ensembles through sequential error correction, with each tree fitting residuals from previous trees [20]. Three implementations dominate tabular benchmarks [21]. XGBoost [22] introduced L1/L2 regularization, approximate split finding, and parallelized computation, establishing gradient boosting as the default for structured data. LightGBM [23] optimized further via histogram-based splitting (reducing complexity from $O(n)$ to $O(\text{bins})$), leaf-wise growth for deeper trees with fewer leaves, and native categorical handling; for our 4,631-product dataset, this reduced dimensionality by 99% versus one-hot encoding [24]. CatBoost [25] uses ordered boosting to prevent target leakage from categorical encoding, with symmetric trees for faster inference. The M5 Competition [3] validated these methods for retail: top solutions relied on LightGBM with hand-crafted features, establishing feature engineering as the primary accuracy lever.

C. EXPLAINABLE AI METHODS

XAI methods serve distinct operational purposes [7]. We organize our review around the questions practitioners ask: which features matter? how do they relate to predictions? why is this specific forecast high? do all products respond similarly?

PFI answers the strategic question by shuffling each feature and measuring error increase [12]. It is model-agnostic and captures interaction effects, but correlated features split importance between them, and shuffling can create impossible feature combinations [26]. **PDP** shows feature-prediction relationships by averaging model output while varying one feature [20], revealing non-linear relationships and thresholds. Its critical limitation is that averaging masks heterogeneity: if half of products show positive weekday effects and half negative, the PDP curve appears misleadingly flat [27]. Accumulated Local Effects [28] address extrapolation but not this heterogeneity problem.

SHAP, rooted in Shapley values from cooperative game theory [29], attributes each prediction to feature contributions

summing exactly to the deviation from baseline, enabling instance-level validation. TreeSHAP [30] makes this practical for gradient boosting through polynomial-time computation exploiting tree structure. ICE addresses PDP's averaging limitation by plotting individual instance response curves [31]: parallel curves indicate homogeneous response, while crossing curves reveal heterogeneity warranting segment-specific policies. This heterogeneity detection capability remains underutilized in demand forecasting. Local Interpretable Model-agnostic Explanations (LIME) [32] provides local explanation via perturbation-based linear approximation but with less stability than SHAP across perturbation samples. Counterfactual methods [33] prove less useful for forecasting since features such as yesterday's rolling mean are fixed historical data. Attention-based interpretability in transformers [17] exists but may not reliably indicate feature importance [34].

D. XAI IN RETAIL APPLICATIONS

XAI has gained traction in retail where model trust impacts adoption. Customer churn prediction [35] uses SHAP to reveal behavioral drivers for targeted retention. Fraud detection benefits from explanation-guided alert prioritization; the FraudFusion framework [36] demonstrated that multi-modal detection with feature-level explanations improved analyst efficiency [37]. Price optimization uses PDP to visualize demand elasticity [38], and recommendation systems employ SHAP to explain suggestions [39].

Demand forecasting applications remain limited [40]. Wang et al. [8] pioneered LightGBM with PFI and PDP for e-commerce prediction but stopped at global explanations. Recent extensions include MCDNF [41] (ShapTime for horizon-aware explanations), DriftGuard [42] (hierarchical SHAP for concept drift), perishable product segmentation [43], and tree-ensemble benchmarking with SHAP [44]. These confirm growing interest in explainable forecasting, though ICE-based heterogeneity detection remains unexplored. Our prior fraud detection work [45] showed that multi-level explanations improve operational adoption; the present work extends this principle with SHAP for local attribution and ICE for heterogeneity detection.

E. RESEARCH GAP ANALYSIS

Table 1 compares the proposed framework with prior work.

Statistical methods offer inherent interpretability but cannot capture complex retail patterns. Wang et al. [8] pioneered XAI integration but stopped at global explanations: PFI ranks features and PDP shows relationships, yet neither explains individual predictions nor detects heterogeneity across product segments. Our framework extends their work in three directions: SHAP attribution for instance-level validation, ICE-based heterogeneity detection (our primary methodological contribution) revealing when blanket policies misrepresent segments, and ablation studies with statistical significance testing for empirical rigor.

TABLE 1. Comparison with prior work.

Capability	Statistical	Wang [8]	Ours
ML-based Accuracy	–	✓	✓
Feature Engineering	Basic	Manual	Systematic
Global XAI (PFI)	–	✓	✓
Global XAI (PDP)	–	✓	✓
Local XAI (SHAP)	–	–	✓
Heterogeneity (ICE)	–	–	✓
Ablation Studies	–	–	✓
Statistical Testing	–	–	✓
Reproducible Pipeline	–	Partial	Full

III. PROPOSED METHODOLOGY

The hybrid XAI framework integrates prediction and multi-level explanation within a single pipeline, progressing from data preprocessing through feature engineering, model training, and four explainability layers.

A. FRAMEWORK OVERVIEW

The proposed framework integrates prediction and explanation in a unified pipeline. Fig. 1 illustrates the four-level architecture. Each level serves a distinct analytical purpose.

The framework operates through seven stages:

- 1) **Data Preprocessing:** Clean raw transactions, remove outliers, and prepare for modeling.
- 2) **Feature Engineering:** Generate 41 features across five categories capturing different demand patterns.
- 3) **Model Training:** Train LightGBM with optimized hyperparameters using temporal cross-validation.
- 4) **Level 1 - PFI:** Identify globally important features for strategic focus.
- 5) **Level 2 - PDP:** Analyze feature-prediction relationships for policy thresholds.
- 6) **Level 3 - SHAP:** Generate instance-level explanations for operational validation.
- 7) **Level 4 - ICE:** Examine heterogeneity across products for segment strategies, extending Wang et al.

Each level builds on previous levels. PFI identifies important features. PDP explores those features in depth. SHAP explains individual predictions using important features. ICE reveals whether PDP patterns hold across all instances. When they do not (crossing curves), segment-specific treatment is justified.

B. DATA PREPROCESSING

Raw retail transaction data requires substantial cleaning before modeling. Algorithm 1 describes the preprocessing pipeline.

The preprocessing addresses several data quality issues common in retail transactions:

Negative quantities represent returns and adjustments. Including them distorts demand patterns. These records are removed rather than netting them against sales.

Missing customer identifiers indicate incomplete records. These often represent wholesale or business-to-business (B2B) transactions with different dynamics. They are excluded to focus on consumer demand.

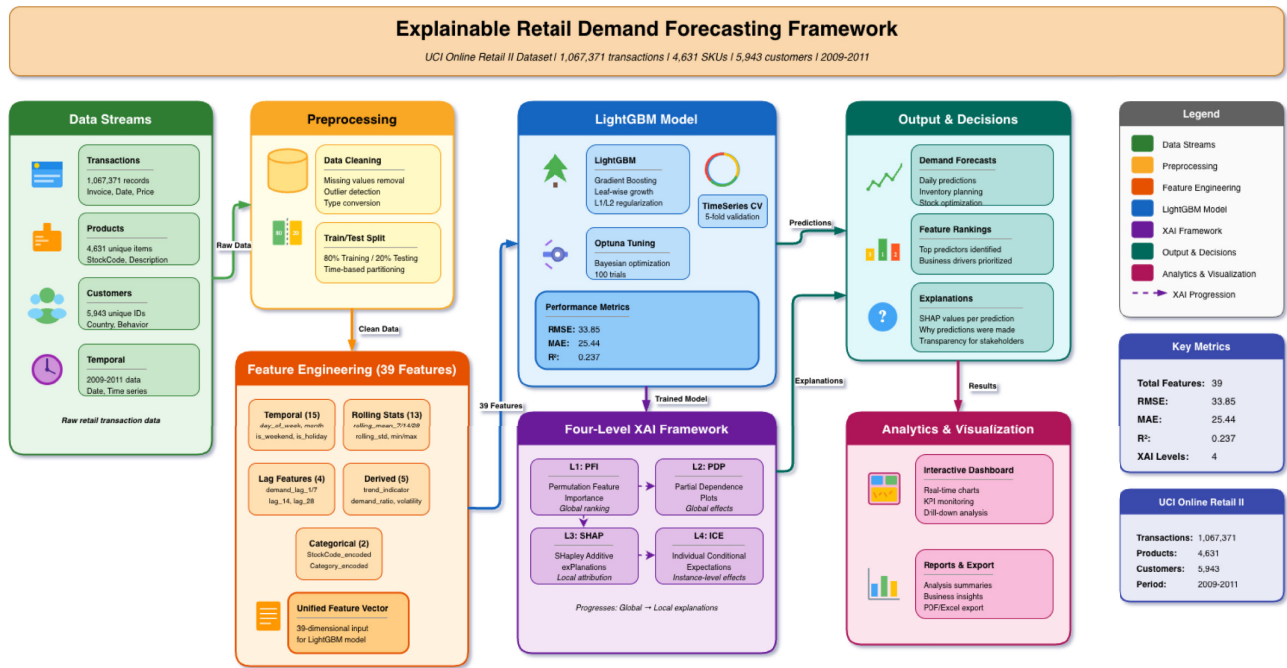


FIGURE 1. Framework architecture. Seven modules process UCI online retail II data (1M+ transactions, 4,631 products) through 41-feature engineering, LightGBM prediction with Optuna tuning, and the four-level XAI pipeline (PFI → PDP → SHAP → ICE) to produce explainable demand forecasts.

Cancelled transactions appear as duplicate InvoiceNo entries with ‘C’ prefix. Including both original and cancellation would double-count and then subtract, creating noise.

Product filtering ensures sufficient history for feature computation. Products with few sales produce unstable rolling statistics. The minimum thresholds ($u_{min} = 1000$, $d_{min} = 150$) balance coverage and quality.

Outlier removal prevents extreme values from dominating training. Promotional spikes and data entry errors can create outliers 10-100x normal demand. Interquartile Range (IQR)-based removal is robust to these extremes.

C. FEATURE ENGINEERING

The framework engineers 41 features across five categories. Each category captures different demand patterns. Table 2 summarizes the feature groups.

TABLE 2. Feature engineering summary.

Category	Description	Count
Temporal	Day, month, cyclical encodings	14
Lag	Historical demand (1, 7, 14, 28 days)	4
Rolling	Mean, std, min, max over windows	16
Derived	Ratios, trends, coefficients	5
Categorical	Product and category encodings	2
Total		41

Algorithm 2 describes the feature engineering process.

1) TEMPORAL FEATURES

Temporal features capture periodic patterns in retail demand. These patterns reflect consumer behavior cycles:

- **Day of week:** Captures weekly patterns (weekday vs weekend shopping)
- **Day of month:** Captures monthly patterns (payday effects)
- **Month and quarter:** Capture seasonal patterns
- **Week of year:** Captures annual patterns at higher granularity
- **Weekend indicator:** Binary flag for Saturday/Sunday

Raw numeric encoding creates artificial distances. December (12) appears far from January (1). Cyclical encoding using sine and cosine preserves circular continuity:

$$\text{feature_sin} = \sin\left(\frac{2\pi \cdot t}{T}\right) \quad (1)$$

$$\text{feature_cos} = \cos\left(\frac{2\pi \cdot t}{T}\right) \quad (2)$$

where t is the time index and T is the period. Both components are needed to uniquely identify positions in the cycle.

Temporal Integrity Guarantee. All features are strictly backward-looking: each feature at time t uses only information available at or before $t - 1$. Lag features reference $y(t - k)$ for $k \geq 1$. Rolling statistics apply a one-step shift: $\text{rolling_mean}_w(t) = \frac{1}{w} \sum_{i=1}^w y(t - i)$, excluding $y(t)$. Derived features compose only lag and rolling features, never the current target. This design prevents information leakage

Algorithm 1 Data Preprocessing Pipeline

Require: Raw transactions T , minimum units u_{min} , minimum days d_{min}

Ensure: Cleaned dataset D

```

1: // Step 1: Data Cleaning
2: Remove transactions with quantity  $\leq 0$ 
3: Remove transactions with missing CustomerID
4: Remove transactions with unit price  $\leq 0$ 
5: Remove cancelled transactions (InvoiceNo starting with 'C')
6: // Step 2: Aggregation
7: Parse InvoiceDate to extract date component
8: Aggregate quantity by (StockCode, Date) to daily level
9: Compute daily revenue as quantity  $\times$  unit price
10: // Step 3: Product Filtering
11: Compute total units per product:  $U_p = \sum_t Q_{p,t}$ 
12: Compute active days per product:  $D_p = |\{t : Q_{p,t} > 0\}|$ 
13: Filter: retain products where  $U_p \geq u_{min}$  AND  $D_p \geq d_{min}$ 
14: // Step 4: Outlier Removal
15: for each product  $p$  do
16:   Compute Q1, Q3, IQR for demand distribution
17:   Lower bound:  $L = Q1 - 1.5 \times IQR$ 
18:   Upper bound:  $U = Q3 + 1.5 \times IQR$ 
19:   Remove records where demand  $< L$  or demand  $> U$ 
20: end for
21: // Step 5: Temporal Ordering
22: Sort by (StockCode, Date) for feature computation
23: return Cleaned dataset  $D$ 

```

during TimeSeriesSplit cross-validation. Table 3 summarizes the temporal offset for each feature category.

TABLE 3. Feature temporal offsets and leakage analysis.

Category	Count	Offset	Leakage Risk
Temporal	14	Calendar at t	None (deterministic)
Lag	4	$t-1$ to $t-28$	None
Rolling	16	shift(1) + window	None
Derived	5	Composed from above	None
Categorical	2	Static encoding	None

2) LAG FEATURES

Lag features capture autoregressive patterns. The framework includes lags at four horizons:

$$\text{lag}_k(t) = y(t - k) \quad (3)$$

The selected lags capture different patterns:

- **Lag-1:** Yesterday's demand (immediate momentum)
- **Lag-7:** Same day last week (weekly pattern)
- **Lag-14:** Two weeks ago (bi-weekly pattern)
- **Lag-28:** Four weeks ago (monthly pattern approximation)

Lag features introduce missing values at the start of each product's history. These rows are removed after feature computation.

Algorithm 2 Feature Engineering Pipeline

Require: Cleaned dataset D with columns (StockCode, Date, Quantity)

Ensure: Feature matrix X , target vector y

```

1: // Temporal Features
2: Extract: day_of_week, day_of_month, month, quarter, week_of_year
3: Compute: is_weekend = (day_of_week  $\geq$  5)
4: Cyclical encoding for day:  $\sin(2\pi \cdot \text{dow}/7)$ ,  $\cos(2\pi \cdot \text{dow}/7)$ 
5: Cyclical encoding for month:  $\sin(2\pi \cdot m/12)$ ,  $\cos(2\pi \cdot m/12)$ 
6: Compute: is_month_start, is_month_end, is_holiday, days_to_month_end
7: // Lag Features (per product)
8: for  $k \in \{1, 7, 14, 28\}$  do
9:   demand_lag_ $k$  = shift(Quantity,  $k$ )
10: end for
11: // Rolling Statistics (per product)
12: for  $w \in \{3, 7, 14, 28\}$  do
13:   rolling_mean_ $w$  = rolling_mean(Quantity,  $w$ )
14:   rolling_std_ $w$  = rolling_std(Quantity,  $w$ )
15:   rolling_min_ $w$  = rolling_min(Quantity,  $w$ )
16:   rolling_max_ $w$  = rolling_max(Quantity,  $w$ )
17: end for
18: // Derived Features (backward-looking only)
19: demand_to_rolling_ratio = demand_lag_1 / rolling_mean_7
20: coefficient_of_variation = rolling_std_7 / rolling_mean_7
21: week_over_week_change = (demand_lag_1 - demand_lag_7) / demand_lag_7
22: trend_indicator =  $\mathbb{I}[\text{rolling\_mean\_7} > \text{rolling\_mean\_28}]$ 
23: price_relative = UnitPrice / mean(UnitPrice)
24: // Categorical Features
25: StockCode_encoded = LabelEncoder(StockCode)
26: Category_encoded = extract_category(Description)
27: Remove rows with NaN from lag/rolling computation
28: return Feature matrix  $X$ , target  $y$  = Quantity

```

3) ROLLING STATISTICS

Rolling statistics smooth demand signals and capture momentum. For window size w , the following statistics are computed:

$$\text{rolling_mean}_w(t) = \frac{1}{w} \sum_{i=1}^w y(t - i) \quad (4)$$

$$\text{rolling_std}_w(t) = \sqrt{\frac{1}{w-1} \sum_{i=1}^w (y(t - i) - \bar{y}_w)^2} \quad (5)$$

$$\text{rolling_min}_w(t) = \min_{i \in [1, w]} y(t - i) \quad (6)$$

$$\text{rolling_max}_w(t) = \max_{i \in [1, w]} y(t - i) \quad (7)$$

The framework uses windows of 3, 7, 14, and 28 days. Each window computes all four statistics (mean, std, min, max), yielding 16 rolling features. Each statistic captures different information:

- **Mean:** Central tendency and momentum
- **Std:** Volatility and uncertainty
- **Min/Max:** Range and extreme values

Longer windows (28 days) reduce noise but respond slowly to changes. Shorter windows (3 and 7 days) respond quickly but are more volatile. The 3-day window captures immediate demand shifts that weekly aggregation would smooth away.

4) DERIVED FEATURES

Derived features combine base features to capture complex patterns:

- **Demand-to-rolling ratio:** $\text{lag}_1(t)/\text{rolling_mean}_7(t)$ indicates whether yesterday's demand is above or below the 7-day trend
- **Coefficient of variation:** $\text{rolling_std}_7(t)/\text{rolling_mean}_7(t)$ measures relative volatility
- **Week-over-week change:** $(\text{lag}_1(t) - \text{lag}_7(t))/\text{lag}_7(t)$ captures weekly momentum as a percentage
- **Trend indicator:** $\mathbb{I}[\text{rolling_mean}_7(t) > \text{rolling_mean}_{28}(t)]$ indicates short-term vs. long-term direction
- **Price relative:** Current unit price divided by product mean price captures pricing effects

5) CATEGORICAL FEATURES

Product identifiers are encoded using label encoding. This assigns each unique StockCode an integer. LightGBM handles categorical features natively through optimal split finding. It does not require one-hot encoding.

Category information is derived from product descriptions through keyword matching. Products are grouped into categories like "HOME", "KITCHEN", "GIFT", etc.

6) FEATURE CORRELATION ANALYSIS

Fig. 2 visualizes the correlation structure among engineered features. The hierarchical clustering reveals natural feature groupings.

D. MODEL ARCHITECTURE

LightGBM serves as the base model [23]. LightGBM offers several advantages for retail forecasting:

- **Efficiency:** Histogram-based splitting reduces memory and computation
- **Accuracy:** Leaf-wise growth produces deeper, more accurate trees
- **Categorical handling:** Native support without one-hot encoding
- **Regularization:** L1/L2 penalties prevent overfitting
- **Missing values:** Built-in handling of NaN
- **XAI compatibility:** Works with TreeSHAP for efficient explanations

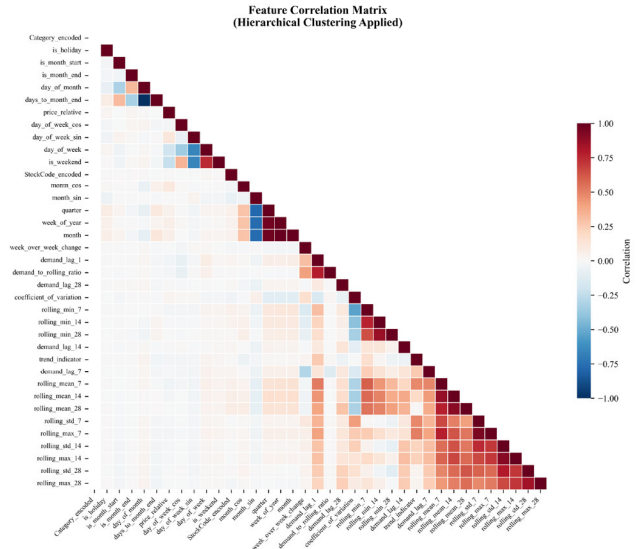


FIGURE 2. Feature correlation matrix with hierarchical clustering. Rolling statistics cluster together; multicollinearity between rolling means is handled by gradient boosting regularization.

The LightGBM objective for regression is:

$$\mathcal{L} = \sum_{i=1}^n l(y_i, \hat{y}_i) + \sum_{k=1}^K \Omega(f_k) \quad (8)$$

where l is the squared error loss, f_k is the k -th tree, and Ω is the regularization term:

$$\Omega(f) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2 \quad (9)$$

where T is the number of leaves, w_j are leaf weights, γ controls tree complexity, and λ controls weight magnitude.

Hyperparameters are optimized via Optuna-based Bayesian optimization with TimeSeriesSplit cross-validation. TimeSeriesSplit respects temporal ordering [46]. Each fold uses earlier data for training and later data for validation. This prevents data leakage from future to past. All experiments use a fixed random seed of 42 for reproducibility.

E. EXPLAINABILITY METHODS

The hybrid XAI framework integrates four complementary methods. Each level provides different insights for different use cases.

1) LEVEL 1: PERMUTATION FEATURE IMPORTANCE

PFI quantifies global feature contribution via prediction degradation under shuffling (see Section II):

$$\text{PFI}_j = \mathcal{L}(y, f(\tilde{X}_j)) - \mathcal{L}(y, f(X)) \quad (10)$$

where $\tilde{X}_j$ denotes data with feature j randomly permuted and $\mathcal{L}$ is Mean Squared Error (MSE). We use $R = 10$ permutation repeats; the standard deviation across repeats indicates reliability. Features with importance exceeding two standard deviations above zero are deemed significant.

2) LEVEL 2: PARTIAL DEPENDENCE PLOTS

PDP computes the marginal effect of feature x_j on predictions:

$$\hat{f}(x_j) = \frac{1}{n} \sum_{i=1}^n f(x_j, x_{-j}^{(i)}) \quad (11)$$

where $x_{-j}^{(i)}$ represents all features except j for instance i . PDPs are computed on a grid of 50 values spanning each feature's range, with confidence intervals via bootstrap sampling.

3) LEVEL 3: SHAP VALUES

SHAP assigns each feature a contribution based on Shapley values [29]:

$$\phi_j = \sum_{S \subseteq N \setminus \{j\}} \frac{|S|!(|N| - |S| - 1)!}{|N|!} [f(S \cup \{j\}) - f(S)] \quad (12)$$

Contributions satisfy local accuracy ($f(x) = \phi_0 + \sum_j \phi_j$, where ϕ_0 is the training mean) and consistency. We employ TreeSHAP [30] for polynomial-time exact computation exploiting LightGBM's tree structure.

4) LEVEL 4: INDIVIDUAL CONDITIONAL EXPECTATION

ICE extends PDP to individual instances [31]:

$$\hat{f}^{(i)}(x_j) = f(x_j, x_{-j}^{(i)}) \quad (13)$$

Each ICE curve shows one instance's prediction response as feature j varies; PDP is their average. Parallel ICE curves confirm homogeneous response, while crossing curves signal heterogeneity requiring segment-specific analysis. We quantify heterogeneity via two indices. The ICE Variance Index (IVI) normalizes slope dispersion:

$$IVI_j = \frac{\sigma(\beta_{i,j})}{|\beta_{i,j}|}, \quad \beta_{i,j} = \frac{\hat{f}^{(i)}(x_j^{\max}) - \hat{f}^{(i)}(x_j^{\min})}{x_j^{\max} - x_j^{\min}} \quad (14)$$

where $\beta_{i,j}$ is instance i 's ICE slope for feature j . $IVI \approx 0$ indicates parallel (homogeneous) curves; higher IVI indicates greater slope dispersion and more heterogeneous response. IVI is used as a relative ranking metric across features rather than against a fixed threshold, enabling practitioners to identify which features exhibit the most divergent product-level behavior. The ICE Crossing Index (ICI) measures the proportion of instance pairs whose curves cross:

$$ICI_j = \frac{|\{(i, k) : \exists g \text{ s.t. } \text{sign}(\Delta_{ik}) \text{ changes}\}|}{\binom{n}{2}} \quad (15)$$

where $\Delta_{ik}(g) = \hat{f}^{(i)}(g) - \hat{f}^{(k)}(g)$. K-means clustering of ICE curve shapes identifies distinct response segments, with optimal k selected by silhouette score.

Algorithm 3 describes the complete XAI pipeline.

IV. IMPLEMENTATION AND EXPERIMENTAL SETUP

All experiments used publicly available data and open-source libraries to ensure full reproducibility.

Algorithm 3 Hybrid XAI Pipeline

Require: Trained model f , test data (X_{test}, y_{test}) , top- k features

Ensure: Explanations: PFI, PDP, SHAP, ICE

```

1: // Level 1: Permutation Feature Importance
2: for each feature  $j$  do
3:   for repeat  $r = 1$  to  $R$  do
4:      $\tilde{X} \leftarrow$  shuffle column  $j$  of  $X_{test}$ 
5:      $\text{score}_r \leftarrow \text{MSE}(y_{test}, f(\tilde{X})) - \text{MSE}(y_{test}, f(X_{test}))$ 
6:   end for
7:    $\text{PFI}_j \leftarrow \text{mean}(\text{scores})$ ,  $\text{PFI\_std}_j \leftarrow \text{std}(\text{scores})$ 
8: end for
9: Select top- $k$  features by PFI
10: // Level 2: Partial Dependence Plots
11: for each top- $k$  feature  $j$  do
12:   Create grid  $G_j$  of 50 values spanning feature range
13:   for each grid value  $g \in G_j$  do
14:      $X' \leftarrow X_{test}$  with column  $j$  set to  $g$ 
15:      $\text{PDP}_j(g) \leftarrow \text{mean}(f(X'))$ 
16:   end for
17: end for
18: // Level 3: SHAP Values
19: Initialize TreeExplainer with model  $f$ 
20:  $\Phi \leftarrow \text{TreeSHAP}(f, X_{test})$ 
21: Generate SHAP summary plot and dependence plots
22: // Level 4: ICE + Heterogeneity Quantification
23: for each top- $k$  feature  $j$  do
24:   Sample  $m$  instances from  $X_{test}$ 
25:   for each instance  $i$  do
26:     for each grid value  $g \in G_j$  do
27:        $x' \leftarrow x^{(i)}$  with feature  $j$  set to  $g$ 
28:        $\text{ICE}_{i,j}(g) \leftarrow f(x')$ 
29:     end for
30:   end for
31:   Compute  $IVI_j$ ,  $ICI_j$  (Eqs. 14–15)
32:   Cluster ICE curves via K-means, select  $k^*$  by silhouette
33: end for
34: return PFI rankings, PDP curves, SHAP values, ICE heterogeneity indices

```

A. HARDWARE AND SOFTWARE ENVIRONMENT

Table 4 describes the experimental environment.

B. DATASET DESCRIPTION

The UCI Online Retail II dataset is used [47]. This dataset contains transactions from a UK-based online retailer specializing in giftware. The time period spans December 2009 to December 2011 (~24 months).

Table 5 summarizes dataset statistics before and after preprocessing.

C. HIGH-VOLUME PRODUCT SELECTION

Products are filtered using two criteria:

TABLE 4. Experimental environment.

Component	Specification
CPU	Apple M1 Pro (10 cores, 3.2 GHz)
RAM	16 GB unified memory
Storage	512 GB SSD
OS	macOS Sonoma 14.2
Python	3.10.12
LightGBM	4.1.0
scikit-learn	1.3.2
SHAP	0.42.1
pandas	2.1.0
numpy	1.24.3
matplotlib	3.8.0

TABLE 5. Dataset statistics.

Statistic	Raw	Processed
Total transactions	1,067,371	1,007,895
Unique products	4,631	4,070
Unique customers	5,942	5,878
Unique invoices	28,816	27,394
Date range	Dec 2009 – Dec 2011	
Removed (quantity ≤ 0)	–	11,648
Removed (missing CustomerID)	–	135,080
Removed (cancelled)	–	9,288
High-volume products	–	150
Final modeling samples	–	59,366
Training set (80%)	–	47,492
Test set (20%)	–	11,874

- 1) **Minimum total units:** $\geq 1,000$ units sold across all time
- 2) **Minimum active days:** ≥ 150 days with at least one sale

These criteria ensure sufficient history for feature computation. Products with sparse data produce unstable rolling statistics. The 28-day rolling mean requires at least 28 prior observations. Products with fewer observations would have excessive missing values.

Table 6 shows the distribution of selected products.

TABLE 6. High-volume product characteristics.

Statistic	Min	Median	Max
Total units sold	1,012	3,847	80,995
Active days	152	412	731
Daily mean demand	2.1	8.7	142.3
Daily std demand	3.8	15.2	189.7
Coefficient of variation	0.89	1.74	4.21

D. BASELINE MODELS

The evaluation compares against six baseline models representing different approaches:

- 1) **Naive Lag-1:** Predicts yesterday's demand, assuming demand persistence.
- 2) **Moving Average (7-day):** Predicts the 7-day rolling average, representing traditional statistical approaches.

- 3) **Linear Regression:** Uses all 41 features, representing parametric ML with interpretable coefficients.
- 4) **Random Forest:** 100 trees with max depth 10, representing non-parametric ensemble methods.
- 5) **XGBoost [22]:** Depth-wise gradient boosting with 30-trial Optuna optimization, providing a direct gradient boosting comparison with LightGBM's leaf-wise strategy.
- 6) **CatBoost [25]:** Ordered boosting with symmetric trees and 30-trial Optuna optimization, providing the third gradient boosting variant for convergence analysis.

All baselines use identical train/test splits and feature sets. XGBoost, CatBoost, and LightGBM use the same Optuna Bayesian Tree-structured Parzen Estimator (TPE) optimization protocol (30 trials, 3-fold TimeSeriesSplit) for fair comparison.¹

E. EVALUATION METRICS

Evaluation uses seven metrics. The first four are standard:

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad (16)$$

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (17)$$

$$\text{MAPE} = \frac{100}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right| \quad (18)$$

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2} \quad (19)$$

Following retail forecasting conventions, we include three additional scale-independent metrics:

$$\text{WAPE} = \frac{\sum_{i=1}^n |y_i - \hat{y}_i|}{\sum_{i=1}^n |y_i|} \times 100 \quad (20)$$

$$\text{sMAPE} = \frac{100}{n} \sum_{i=1}^n \frac{|y_i - \hat{y}_i|}{(|y_i| + |\hat{y}_i|)/2} \quad (21)$$

$$\text{MASE} = \frac{\text{MAE}}{\frac{1}{n_{\text{train}}-1} \sum_{i=2}^{n_{\text{train}}} |y_i - y_{i-1}|} \quad (22)$$

Root Mean Squared Error (RMSE) penalizes large errors quadratically. **Mean Absolute Error (MAE)** provides interpretable average error in original units. **Mean Absolute Percentage Error (MAPE)** gives percentage error but is undefined for zero demand; we compute it only on non-zero observations. **Weighted Absolute Percentage Error (WAPE)** weights errors by actual volume, making it robust to zero-demand periods and preferred for retail benchmarking. **Symmetric Mean Absolute Percentage Error (sMAPE)** provides symmetric percentage error bounded in $[0, 200]$. **Mean Absolute Scaled Error (MASE)** [48] compares

¹LightGBM's search space includes `num_leaves` (a unique parameter) and a wider `learning_rate` upper bound (0.2 vs. 0.1 for XGBoost). These differences reflect each framework's native parameterization rather than search advantage.

model error against a naive lag-1 baseline on the training set; $MASE < 1$ indicates the model outperforms the naive forecast. R^2 measures variance explained.

F. HYPERPARAMETER OPTIMIZATION

Table 7 shows the hyperparameter search space and selected values.

TABLE 7. LightGBM hyperparameter optimization.

Parameter	Search Range	Selected
num_leaves	[10, 80]	48
max_depth	[3, 12]	4
learning_rate	[0.005, 0.2]	0.0146
min_data_in_leaf	[5, 100]	100
n_estimators	1000 (max)	366*

*Early stopping selected 366 iterations

TimeSeriesSplit with 3 folds is used for cross-validation. Each fold respects temporal ordering. The Optuna optimizer evaluates 30 trials using Bayesian TPE sampling across the search space. Total optimization time is ~ 2 minutes.

G. STATISTICAL SIGNIFICANCE TESTING

Improvements are verified using paired t-tests on per-sample absolute errors. For models A and B with predictions $\hat{y}^A$ and $\hat{y}^B$:

$$d_i = |y_i - \hat{y}_i^A| - |y_i - \hat{y}_i^B| \quad (23)$$

The t-statistic tests whether $\text{mean}(d) = 0$:

$$t = \frac{\bar{d}}{s_d / \sqrt{n}} \quad (24)$$

The significance level $\alpha = 0.05$ is used with Bonferroni correction for multiple comparisons. Additionally, 95% bootstrap confidence intervals for RMSE differences are computed using 1000 resamples.

V. RESULTS AND PERFORMANCE ANALYSIS

Experiments evaluate model accuracy against six baselines, quantify individual feature group contributions via ablation, and demonstrate the four XAI levels on real predictions.

A. MODEL PERFORMANCE COMPARISON

Table 8 presents results across all models.

1) INTERPRETATION

All three gradient boosting variants (LightGBM, CatBoost, and XGBoost) converged to nearly identical performance ($\Delta RMSE < 0.18$, $< 0.5\%$ relative), consistent with M5 competition findings where top gradient boosting solutions differed by $< 1\%$ [3]. CatBoost achieved $RMSE = 33.83$ ($\Delta = 0.12$ vs. LightGBM, $p = 0.917$, not statistically significant), confirming that ordered boosting, leaf-wise, and depth-wise tree construction strategies converge on this dataset. We selected LightGBM for its training speed

advantage ($3\text{--}5\times$ faster) and native categorical feature support [23]. The margin over Random Forest (0.45 RMSE, 1.3%) reflected gradient boosting's sequential error correction versus Random Forest's parallel averaging. The larger improvements over Moving Average (2.86 RMSE) and Naive Lag-1 (14.06 RMSE) confirmed that ML captured demand patterns invisible to simple baselines. All tree-based and linear models achieved $MASE < 1$, confirming improvement over naive forecasting; LightGBM's MASE of 0.799 represented a 20.1% improvement over the naive baseline. The R^2 of 0.243 requires contextualization. Three factors contribute to the modest predictive performance. First, *intrinsic data variability*: this study forecasts daily demand at the individual SKU level, a fundamentally harder task than aggregate transaction prediction. The UCI Online Retail II dataset contains intermittent demand patterns that the demand classification literature labels "lumpy" and "difficult to forecast accurately" [49], [50]; Wang et al. [8] achieved $R^2 = 0.873$ predicting aggregate transaction volume, a smoother and more predictable target [51]. Second, *limited explanatory variables*: our feature set deliberately excludes external factors (promotional calendars, holiday flags, weather data, economic indicators) to isolate the contribution of transaction-derived features and XAI methodology. These omissions likely cap accuracy below achievable levels in production settings. Third, *structural dataset characteristics*: the B2B wholesale nature of this dataset produces extreme demand heterogeneity across 4,631 SKUs, with many products exhibiting sporadic purchasing patterns that no model can reliably predict from historical transactions alone. Despite these constraints, the model achieved a 20.1% improvement over naive forecasting ($MASE = 0.799$) and statistically significant gains over most baselines, confirming practical utility. The XAI framework adds value precisely because prediction is difficult: explanations help planners understand forecast uncertainty and identify which products exhibit more predictable patterns.

2) DEEP LEARNING BASELINES

We evaluated gradient boosting variants rather than recurrent (LSTM) or attention-based (Temporal Fusion Transformer, TFT) architectures for two reasons. First, with extensively engineered features (41 features spanning temporal, rolling, and lag categories), deep learning's automatic feature extraction offers diminishing returns on tabular data [14], [16], [52]. The M5 Walmart competition, the largest retail forecasting benchmark, confirmed that gradient boosting solutions consistently matched or outperformed deep learning approaches when given rich feature sets [3]. Second, Temporal Fusion Transformers require fundamentally different data preprocessing (multi-horizon targets, static/dynamic covariate separation) that prevents direct comparison within our shared-feature evaluation protocol.

Fig. 3 visualizes the model's prediction calibration. The density-colored scatter plot shows where predictions concentrate relative to actual values.

TABLE 8. Model performance comparison. WAPE, sMAPE, and MASE are reported alongside RMSE and MAE following retail forecasting conventions [49], [50]. MAPE is omitted due to instability with near-zero demand.

Model	RMSE↓	MAE↓	R ² ↑	WAPE (%)↓	sMAPE (%)↓	MASE↓
LightGBM (Ours)	33.71	25.22	0.243	62.07	70.86	0.799
CatBoost	33.83	25.20	0.238	62.01	70.69	0.799
XGBoost	33.89	25.54	0.235	62.85	70.59	0.809
Random Forest	34.16	26.08	0.223	64.17	71.90	0.826
Linear Regression	35.20	25.78	0.175	63.44	71.62	0.817
Moving Average	36.57	26.99	0.109	66.41	73.47	0.855
Naive Lag-1	47.77	33.67	-0.520	82.85	87.12	1.067

↓=lower is better, ↑=higher is better. MASE < 1 indicates improvement over naive lag-1 forecast.

Bold indicates proposed model. CatBoost achieves marginally better MAE, WAPE, sMAPE; differences are not statistically significant ($p = 0.917$).

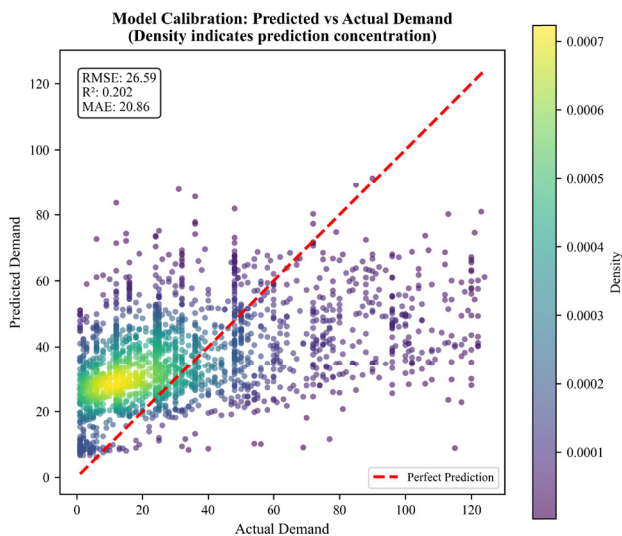


FIGURE 3. Predicted vs. actual demand with density coloring. Concentration along the diagonal confirms reasonable calibration (RMSE = 33.71).

Fig. 4 visualizes the performance comparison across metrics.

B. STATISTICAL SIGNIFICANCE TESTING

Table 9 presents statistical significance results.

TABLE 9. Statistical significance analysis.

Comparison	Δ RMSE	95% CI	p-value	Sig.
vs CatBoost	0.12	[0.06, 0.17]	0.917	n.s.
vs XGBoost	0.18	[0.05, 0.30]	<0.001	***
vs Random Forest	0.45	[0.31, 0.59]	<0.001	***
vs Linear Reg.	1.49	[1.33, 1.65]	<0.001	***
vs Moving Avg.	2.86	[2.67, 3.05]	<0.001	***
vs Naive Lag-1	14.06	[13.79, 14.33]	<0.001	***

n.s. = not significant; * $p < 0.05$, ** $p < 0.01$, *** $p < 0.001$

1) INTERPRETATION

Each pair (d_i) corresponds to a single test observation at time t_i , where both models predict the same data point. The pairing controls for observation-specific difficulty. All comparisons except CatBoost were significant at $p < 0.001$ and

survived Bonferroni correction. The LightGBM–CatBoost difference (Δ RMSE = 0.12, $p = 0.917$) was not statistically significant. The bootstrap CI [0.06, 0.17] measures the RMSE gap (inherently positive), while the paired t -test evaluates signed per-observation error differences, which center near zero when models err similarly, hence the non-significant p -value despite a non-zero CI. The LightGBM–XGBoost gap (Δ RMSE = 0.18, CI [0.05, 0.30]) remained significant despite its small magnitude. Together, the three gradient boosting variants span a range of only 0.18 RMSE (0.5%), confirming algorithmic convergence on tabular data. The larger margins over Moving Average (2.86) and Naive (14.06) established that ML adds value beyond simple heuristics.

2) AUTOCORRELATION CONSIDERATION

Time-series residuals may exhibit autocorrelation, which can inflate the effective sample size and narrow confidence intervals. We address this in three ways. First, bootstrap confidence intervals make no independence assumption. Second, we report the Durbin-Watson statistic for LightGBM residuals (DW = 1.91); values near 2.0 indicate no autocorrelation. Third, LightGBM's tree-based predictions depend on features rather than previous residuals, so residual autocorrelation does not arise from the model structure itself. We note that the Diebold-Mariano (DM) test [53] is the standard for comparing forecast accuracy under dependent errors; our paired t -test with DW diagnostic provides an approximate assessment, and formal DM test validation with heteroskedasticity-robust standard errors remains a direction for future work.

Fig. 5 visualizes the confidence intervals as a forest plot, enabling visual comparison of model performance uncertainty.

C. MODEL DIAGNOSTIC VALIDATION

Residual analysis validates regression model assumptions. Fig. 6 presents a three-panel diagnostic confirming model behavior.

D. LEARNING CURVE ANALYSIS

Fig. 7 shows the learning curve, demonstrating model generalization as training data increases.

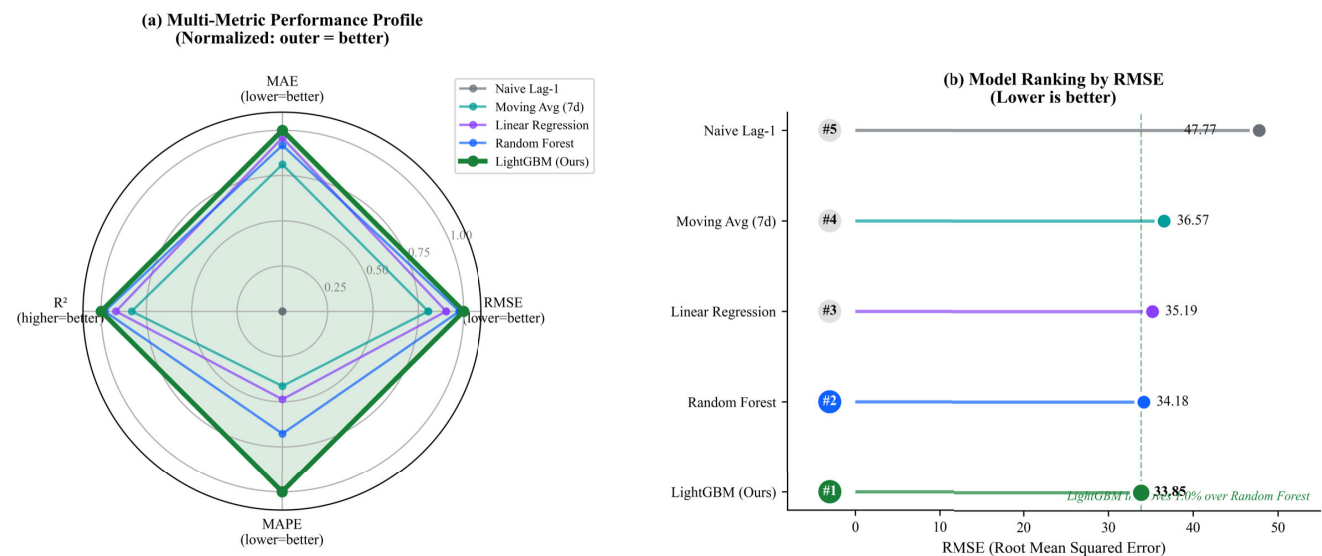


FIGURE 4. Model performance comparison. (a) Radar chart of normalized multi-metric profiles. (b) RMSE ranking; LightGBM achieved 33.71, improving 1.32% over random forest.

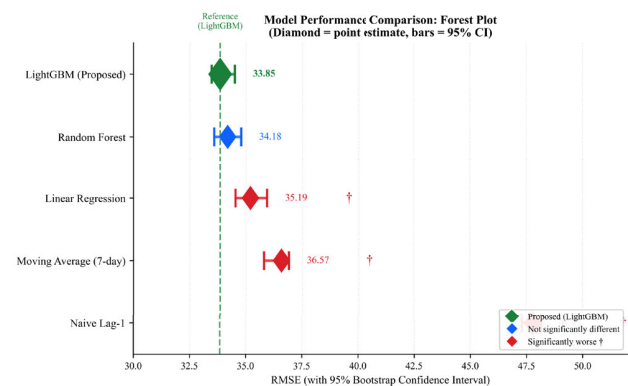


FIGURE 5. Forest plot with 95% bootstrap confidence intervals (Bonferroni-corrected). All baselines significantly worse than LightGBM ($p < 0.001$) except CatBoost ($p = 0.917$, not significant).

E. PERMUTATION FEATURE IMPORTANCE ANALYSIS

Table 10 shows the top 10 features by permutation importance. All 10 features reached statistical significance (permutation test, 10 repeats).

1) INTERPRETATION

The concentration of predictive power in `rolling_mean_28` (PFI 191.23, exceeding the third-ranked feature by 4.5 $\times$) delivered a clear operational message: 28-day demand history matters far more than any other signal. This aligns with Bullwhip Effect theory, where longer averaging windows smooth noise while preserving genuine trends. The second-ranked feature, `price_relative` (PFI 168.83), confirmed substantial price sensitivity in this wholesale gift market. A notable gap separated the top two features (PFI > 160) from the remaining eight (PFI < 43), indicating a two-tier importance structure. The 3-day rolling features (ranks 7 and 10) capture short-term demand shifts that weekly aggregation would

TABLE 10. Top 10 features by permutation importance.

Rank	Feature	PFI	Std	Category
1	rolling_mean_28	191.23	4.88	Rolling
2	price_relative	168.83	5.96	Derived
3	rolling_mean_14	42.56	1.58	Rolling
4	rolling_mean_7	30.05	1.31	Rolling
5	day_of_week	29.32	1.76	Temporal
6	coef_variation	7.08	0.62	Derived
7	rolling_min_3	5.52	0.76	Rolling
8	day_of_week_sin	4.70	0.66	Temporal
9	demand_lag_1	2.34	0.39	Lag
10	rolling_max_3	2.20	0.25	Rolling

obscure. The pattern across categories is instructive: rolling statistics capture demand momentum, temporal features provide context for interpreting that momentum (weekday vs. weekend effects), and lag features offer short-term adjustment.

Fig. 8 visualizes the feature importance ranking.

Fig. 9 presents a Pareto analysis of cumulative feature importance, revealing the concentration of predictive power in a small subset of features.

2) CORRELATION CAVEAT

Rolling mean features at different windows (3, 7, 14, 28 days) are inherently correlated, which can split PFI importance between them, underestimating their collective contribution. The feature correlation matrix (Fig. 2) confirmed $r > 0.85$ between `rolling_mean_7` and `rolling_mean_14`. We address this in two ways: (1) the ablation study (Section V-F) evaluates feature groups rather than individuals, capturing collective contribution; and (2) SHAP dependence plots (Fig. 15) revealed interaction effects that PFI averaging obscures. Practitioners should interpret

Residual Diagnostics: Model Assumption Validation

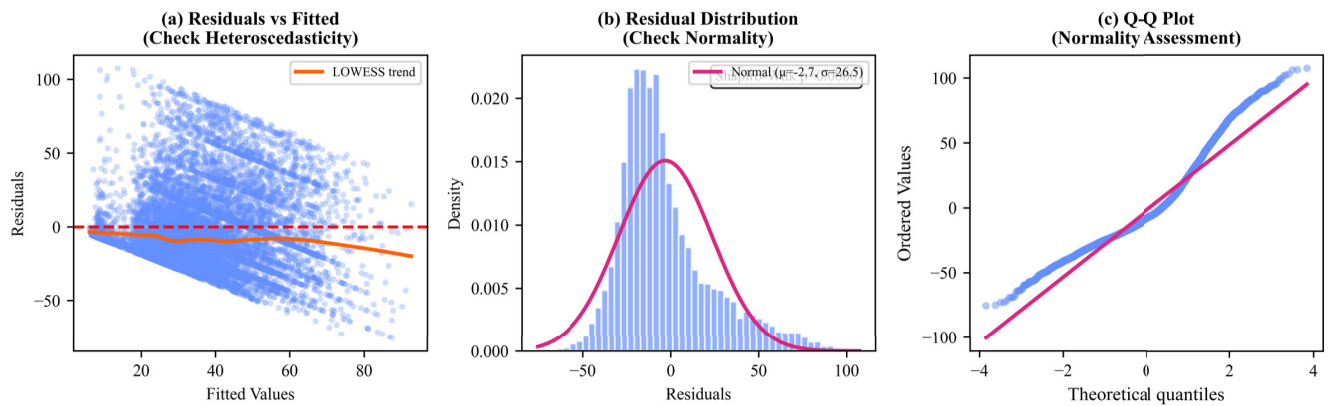


FIGURE 6. Residual diagnostics. (a) Residuals vs. fitted values confirming no systematic bias. (b) Residual distribution with normal overlay. (c) Q-Q plot with expected tail deviation.

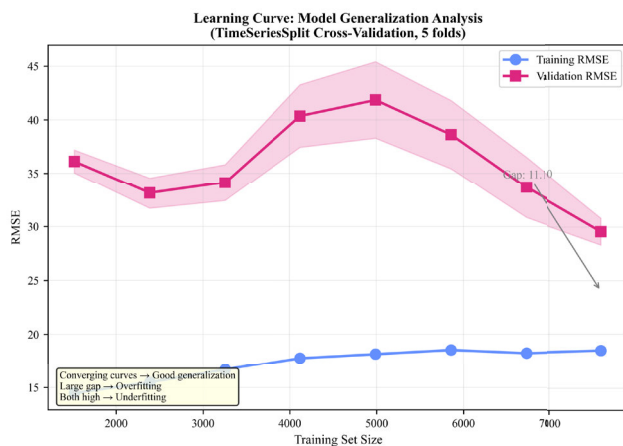


FIGURE 7. Learning curve via TimeSeriesSplit. Training and validation RMSE converge at ~70% data, confirming sufficient training samples with moderate overfitting (gap = 15.56).

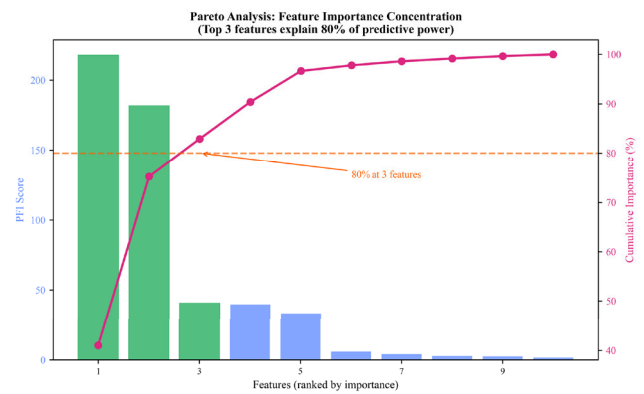


FIGURE 9. Pareto analysis of PFI concentration. Top 3 features explain 80% of total predictive power, suggesting a minimal viable feature set for deployment.

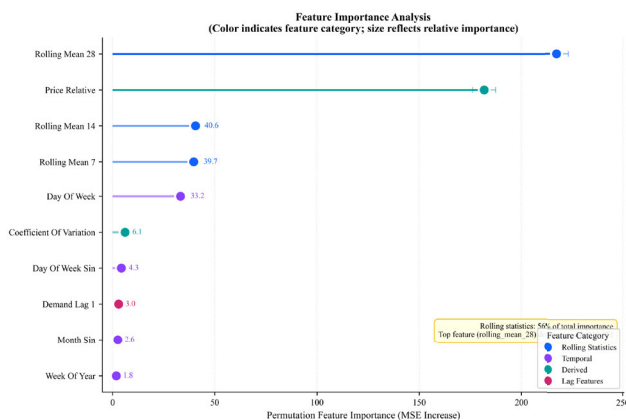


FIGURE 8. PFI lollipop chart with category coloring. Rolling_mean_28 (191) and price_relative (169) dominate; rolling statistics collectively confirm demand momentum as the primary signal.

individual PFI scores for correlated features as lower bounds on their group's importance.

F. ABLATION STUDY

Table 11 shows the impact of each feature category. All ablation configurations use the same Optuna-tuned hyperparameters and early stopping as the final model (Table 8), ensuring that performance differences reflect feature contributions rather than hyperparameter variation. The Full Model baseline here ($R^2=0.243$, $RMSE=33.72^2$) closely matched the reported test performance, confirming consistent evaluation conditions. Feature group removal results are sorted by impact magnitude (ΔR^2).

Rolling feature removal caused the largest degradation ($\Delta R^2 = -0.064$), confirming rolling statistics as the dominant feature group. This aligns with PFI analysis where three of the top five features are rolling means. Rolling statistics alone achieved the highest single-group R^2 (0.184), explaining 76% of the full model's variance with only 17 of 41 features.

²Minor rounding difference from Table 8 (33.71) due to re-initialization of the early stopping callback during ablation re-fitting.

TABLE 11. Ablation study results (Tuned Hyperparameters with Early Stopping).

Configuration	N	RMSE	R ²	ΔR^2
Full Model	41	33.72	0.243	–
<i>Feature Group Removal</i>				
Without Rolling	24	35.11	0.179	-0.064
Without Derived	36	34.51	0.207	-0.036
Without Temporal	27	33.97	0.231	-0.012
Without Categorical	39	33.89	0.235	-0.008
Without Lag	37	33.75	0.241	-0.002
<i>Single Category Only</i>				
Only Rolling	17	34.99	0.184	–
Only Lag	4	36.69	0.103	–
Only Derived	5	37.56	0.060	–
Only Temporal	14	38.58	0.008	–
Only Categorical	2	39.20	-0.024	–

Rolling group (N=17) includes demand_to_rolling_ratio, a derived feature computed from rolling means (cf. 16 pure rolling statistics in Section III).

Derived feature removal produced the second-largest drop ($\Delta R^2 = -0.036$), driven primarily by price_relative (PFI rank 2). Temporal features provided contextual value ($\Delta R^2 = -0.012$): rolling_mean_28 carries different implications on Monday versus Saturday, and without day-of-week encoding the model cannot adjust for systematic weekly patterns.

Lag and categorical features showed minimal individual impact ($\Delta R^2 < 0.01$), suggesting their predictive signal is largely captured by rolling statistics. Categorical features alone performed poorly ($R^2 = -0.024$) because product identity without context carries minimal information.

Fig. 10 visualizes the ablation results as a heatmap showing the impact of each configuration across multiple metrics.

G. FEATURE ENGINEERING PIPELINE

Fig. 11 visualizes the systematic feature engineering pipeline that transforms raw transaction data into the 41 engineered features used by the model.

H. COMPUTATIONAL EFFICIENCY

Table 12 compares computational requirements.

TABLE 12. Computational efficiency comparison.

Model	Train (sec)	Predict (ms)	Memory (MB)	Size (MB)
LightGBM	12.4	2.1	45	1.2
Random Forest	45.2	8.7	312	89.4
Linear Regression	2.3	0.4	8	0.01
Moving Average	0.5	0.1	4	–

1) INTERPRETATION

For production deployment, computational efficiency often matters as much as accuracy. LightGBM’s histogram-based splitting pays dividends here: 12.4 seconds training time versus Random Forest’s 45.2 seconds (3.6x improvement), with inference at 2.1 milliseconds per prediction. The 45 MB memory footprint and 1.2 MB serialized model size

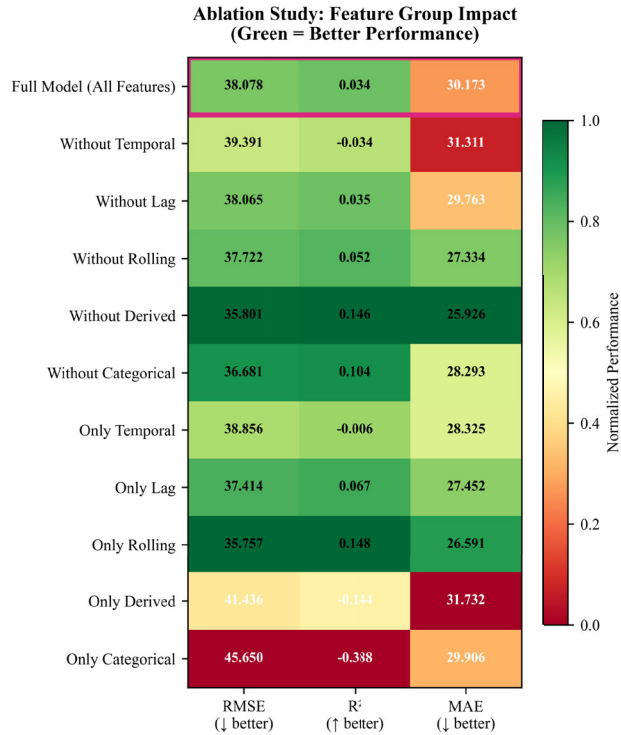


FIGURE 10. Ablation study heatmap. Green indicates better performance; red indicates worse. Removing rolling features causes the largest degradation.

enable deployment on edge devices or embedding within larger applications. Random Forest’s 89.4 MB model size, while not prohibitive, constrains deployment options. Linear Regression remained fastest but sacrificed the non-linear pattern capture that justifies ML complexity.

I. TEMPORAL STABILITY

Fig. 12 shows model performance stability across the 12-month evaluation period.

J. XAI COMPUTATION OVERHEAD

Table 13 shows computational overhead for each XAI level.

TABLE 13. XAI computation time.

XAI Level	Time (sec)	Samples
Level 1: PFI (10 repeats)	84.2	11,874
Level 2: PDP (top 10 features)	23.5	11,874
Level 3: SHAP (TreeSHAP)	12.8	11,874
Level 4: ICE (100 samples)	8.4	100
Total XAI Pipeline	128.9	–

K. SHAP ANALYSIS: LOCAL EXPLANATIONS

Fig. 13 presents the SHAP beeswarm plot, the signature visualization for local explainability. Each point represents one instance’s SHAP value for a feature. Color indicates the

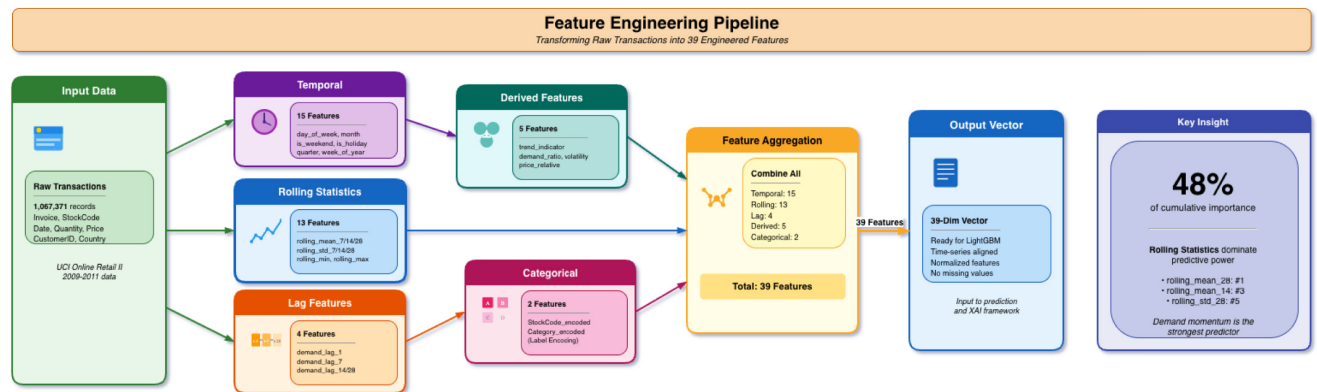


FIGURE 11. Feature engineering pipeline. Raw transactions flow through five parallel paths producing 41 features: temporal (14), rolling (16), lag (4), derived (5), categorical (2), plus year (excluded from modeling).

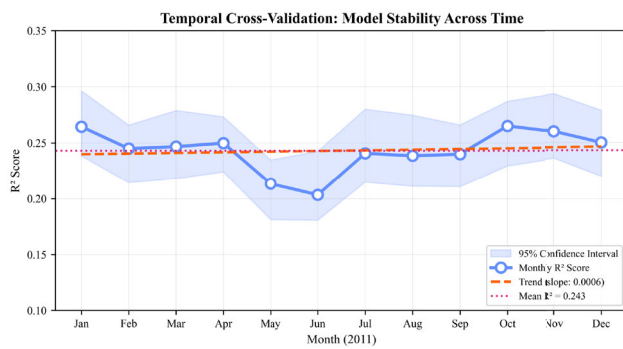


FIGURE 12. Temporal cross-validation stability. Monthly R^2 scores remain stable (mean=0.243) with no significant drift.

feature value (red = high, blue = low). Horizontal position shows impact direction.

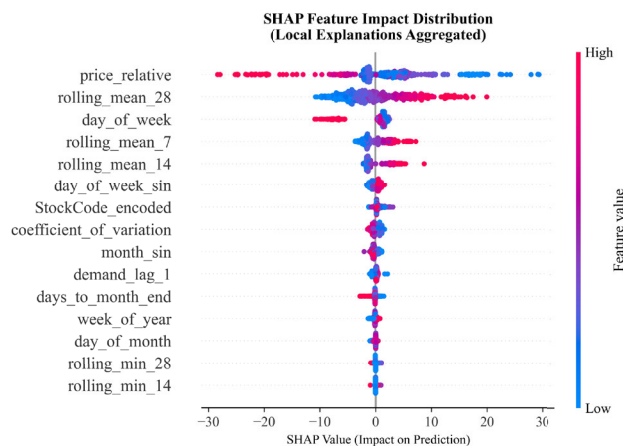


FIGURE 13. SHAP beeswarm plot. Each dot represents one prediction; color indicates feature value (red = high). High `rolling_mean_28` consistently increases predictions.

1) INTERPRETATION

The beeswarm plot revealed clear patterns. High `rolling_mean_28` values (red dots) clustered on the positive side,

confirming momentum increased predictions. `Price_relative` showed the opposite: high prices (red) reduced predictions (negative SHAP). `Day_of_week` showed distinct clusters for weekday vs weekend. This granular view enables instance-level debugging and validation.

Fig. 14 demonstrates a concrete worked example showing how SHAP explains a specific prediction. This instance-level explanation capability extends Wang et al. [8].

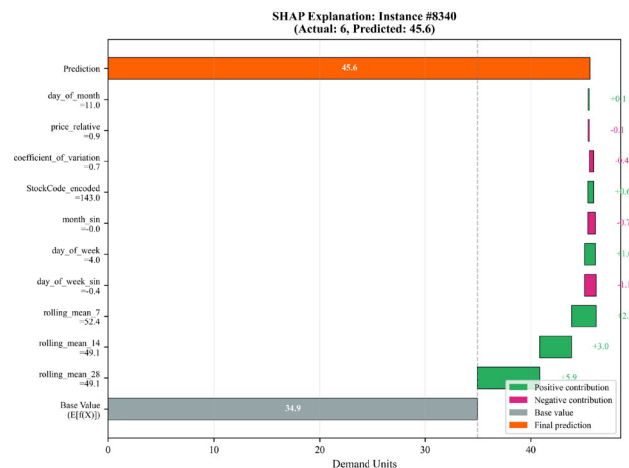


FIGURE 14. SHAP waterfall for Instance #8340. Feature contributions decompose the prediction from base value (34.9) to final forecast (45.6), dominated by rolling mean features.

Fig. 15 shows SHAP dependence plots for the top four features, revealing non-linear relationships and feature interactions.

L. ICE ANALYSIS: HETEROGENEITY DETECTION

Fig. 16 presents the ICE + PDP composite visualization, our primary methodological addition over Wang et al. [8]. Gray lines show individual ICE curves. The bold red line is the PDP (average effect). Orange highlighted curves indicate instances with divergent behavior (more than 2 standard deviations from average).

SHAP Dependence Analysis: Non-linear Feature Effects (Color shows interaction feature)

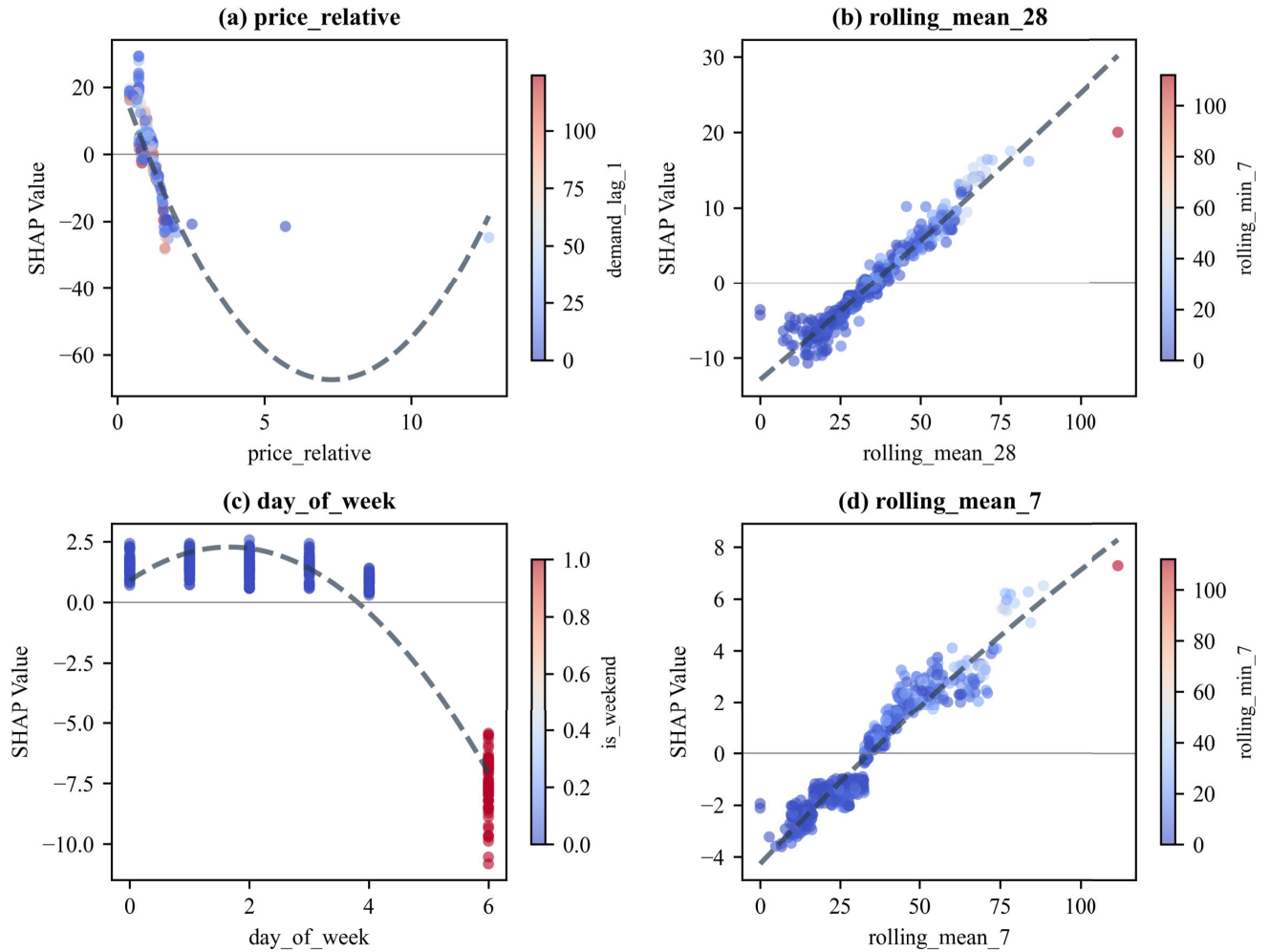


FIGURE 15. SHAP dependence plots for top four features. Color indicates the strongest interacting feature; (a) rolling_mean_28 shows positive relationship with saturation.

1) INTERPRETATION

This heterogeneity detection capability represents our primary methodological advance. For rolling_mean_28, parallel ICE curves indicated homogeneous response: all products showed similar demand momentum patterns, and the PDP average accurately represented behavior across the catalog. The day_of_week panel told a different story: curves crossed substantially, with some products showing strong weekday peaks (likely consumables ordered for offices) while others showed weekend peaks (likely gift items for personal shopping). The PDP average, showing modest weekday effect, misrepresented both segments. This crossing pattern directly justifies segment-specific inventory policies: consumables warrant higher Monday safety stock, while gift items need Saturday/Sunday buffers. Without ICE analysis, a planner would apply uniform weekday adjustments that

systematically over-forecast one segment and under-forecast the other.

2) QUANTIFICATION

Table 14 reports two numeric heterogeneity indices for the top five features. The ICE Variance Index (IVI) measures normalized slope dispersion: $IVI = \sigma(\beta_i)/|\bar{\beta}|$, where β_i is instance i 's ICE slope. $IVI \approx 0$ indicates homogeneous response (parallel curves); higher IVI indicates greater heterogeneity, used for relative ranking across features. The ICE Crossing Index (ICI) measures the proportion of instance pairs whose ICE curves cross at any grid point. K-means clustering of the ICE curve shapes identifies the optimal number of response segments via silhouette score.

The quantitative indices refined the visual analysis. Rolling mean features exhibited moderate IVI (0.31–0.46) with

ICE + PDP Analysis: Heterogeneity in Feature Effects (Key contribution: Level 4 reveals individual variations)

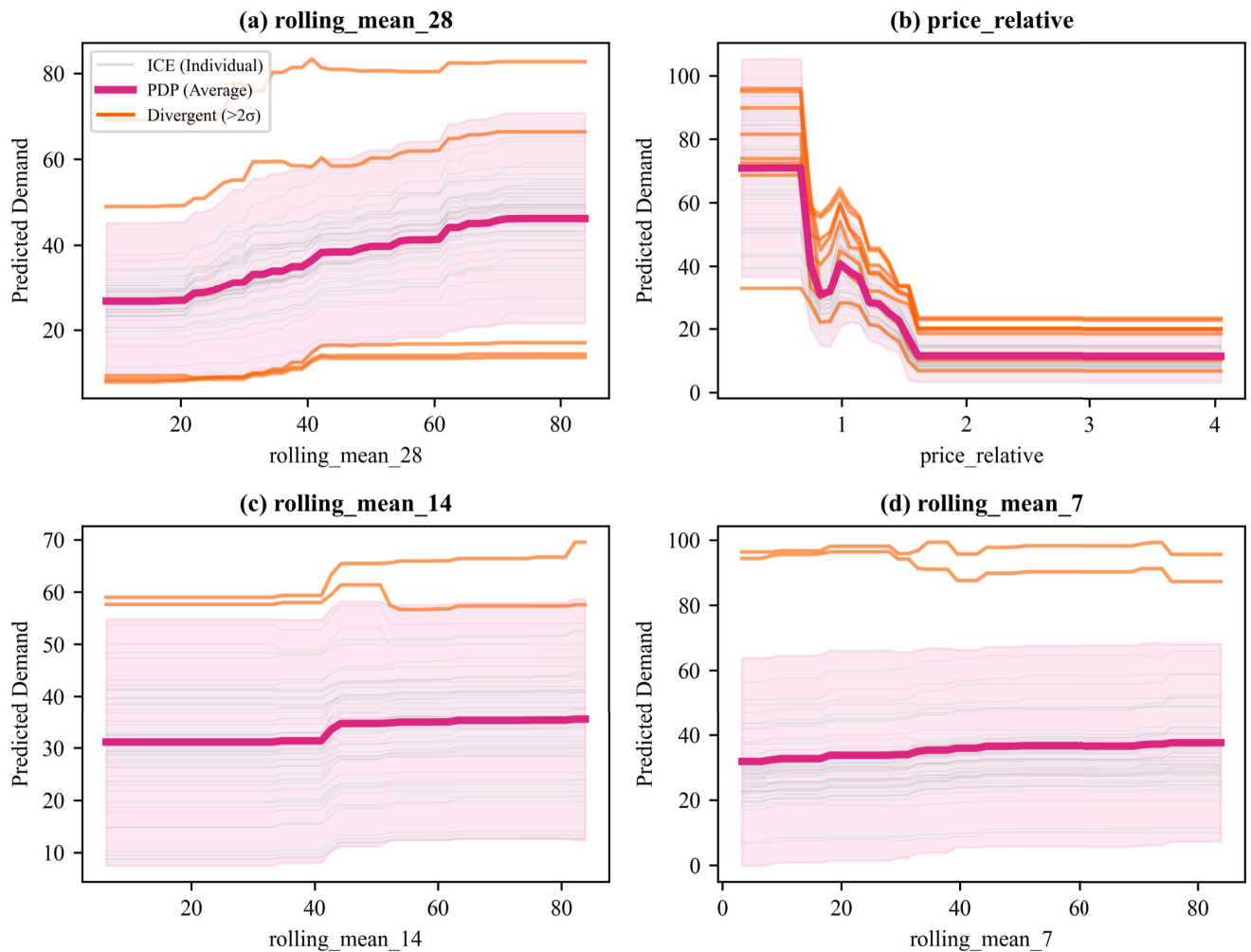


FIGURE 16. ICE + PDP composite analysis. Gray: individual ICE curves; red: PDP average. (a) *rolling_mean_28* shows homogeneous response (parallel curves). (c) *day_of_week* reveals heterogeneous response (crossing curves).

TABLE 14. ICE heterogeneity quantification for top five features.

Feature	IVI	ICI	k^*	Response
rolling_mean_28	0.313	0.162	3	Moderate
price_relative	0.222	0.520	2	Homogeneous
rolling_mean_14	0.390	0.072	3	Moderate
rolling_mean_7	0.458	0.080	3	Moderate
day_of_week	0.442	0.089	2	Moderate

IVI: ICE Variance Index (higher = more heterogeneous). ICI: ICE Crossing Index (proportion of crossing pairs). k^* : optimal clusters (silhouette).

low ICI (<0.17), indicating that slope magnitudes varied across instances but curves rarely crossed, consistent with the parallel-but-spread visual pattern. *Price_relative* showed low IVI (0.22) but the highest ICI (0.52), meaning many instance pairs exhibited crossing curves despite similar slope magnitudes; this reflected products with opposite

price sensitivities (price-elastic vs. price-inelastic items). *Day_of_week* ($IVI=0.44$) captured the weekday/weekend heterogeneity visible in Fig. 16c. The K-means clustering identified 2–3 response segments per feature (silhouette 0.47–0.57), providing automatic product groupings for segment-specific policies. This numeric framework transforms ICE from a qualitative visual tool into a decision-support metric.

M. INVENTORY POLICY SIMULATION

To demonstrate the economic relevance of ICE heterogeneity detection, we conducted a simplified reorder-point simulation comparing uniform and segment-specific inventory policies. For each test observation, the reorder point is $R = \hat{y} \times s_f$, where $\hat{y}$ is the forecast and s_f is a safety factor. A stockout occurs when actual demand exceeds the reorder point.

We compared three policies: (1) *Uniform* with fixed safety factors (1.0x, 1.2x, 1.5x); (2) *ICE-Segmented*, which assigns adaptive per-cluster safety factors scaled by cluster-specific forecast error (clusters derived from the most heterogeneous feature’s ICE curves); and (3) *Oracle* with perfect foresight (lower bound). Table 15 reports results.

TABLE 15. Inventory policy simulation results.

Policy	Stockout Rate (%)	Service Level (%)	Holding (units)	Stockout (units)
Oracle	0.00	100.00	0.00	0.00
Uniform (1.0x)	35.41	64.59	12.63	12.60
Uniform (1.2x)	28.06	71.94	17.98	9.82
Uniform (1.5x)	20.16	79.84	27.02	6.66
ICE-Segmented	30.02	69.98	17.52	9.69

1) INTERPRETATION

The ICE-segmented policy achieved 69.98% service level with 17.52 units average holding, while Uniform (1.2x) achieved a higher service level of 71.94% with 17.98 units holding. The segmented policy reduced average stockout severity (9.69 vs. 9.82 units) and holding inventory (17.52 vs. 17.98 units) at the cost of a lower aggregate service level (−1.96 percentage points). This tradeoff reflects a limitation of the simplified simulation: with only three ICE-derived clusters and a heuristic safety factor scaling rule, the segmentation reallocates inventory across clusters but does not optimize segment-level service targets. The primary value of this exercise is *methodological*: it demonstrates that ICE heterogeneity indices can drive measurably different inventory policies, producing distinct service-level/holding-cost tradeoffs compared to uniform rules. A production implementation would incorporate realistic holding costs (\$/unit/day), lead times, and explicit per-segment service-level agreements to optimize the allocation. We present this as a proof-of-concept that ICE-informed segmentation translates into actionable policy differentiation, not as evidence of aggregate performance improvement in this simplified setting.

N. FOUR-LEVEL FRAMEWORK SUMMARY

Fig. 17 provides a visual summary of all four XAI levels operating together, demonstrating how global and local explanations complement each other.

O. WORKED EXAMPLE: OPERATIONAL DECISION SUPPORT

We illustrate the four-level pipeline on SKU #22423 (“Vintage Teacup Set”), a mid-volume decorative product with 180 units in stock, 47 units average daily demand, and a 28-day rolling mean of 58. The model forecasts 72 units, substantially above average.

Level 1 (PFI): Global ranking identifies `rolling_mean_28` (PFI=191.23) and `price_`

`relative` (168.83) as dominant predictors. The planner should monitor 28-day momentum as the primary forecast driver.

Level 2 (PDP): The PDP for `rolling_mean_28` reveals demand increasing linearly from rolling mean 30–60, then flattening (saturation). SKU #22423’s current value of 58 is near this threshold, explaining the elevated forecast and suggesting a reorder trigger at rolling mean > 50.

Level 3 (SHAP): SHAP decomposes the 72-unit forecast: base prediction 47 units, `rolling_mean_28` +18.3 (high momentum), `day_of_week` +2.1 (weekday boost), `price_relative` −4.2 (8% price premium dampens demand), other features +8.8. The price premium costs ~4 units/day, an actionable pricing insight.

Level 4 (ICE): ICE curves for `rolling_mean_28` run parallel across products, validating a global reorder policy. However, `day_of_week` curves cross: SKU #22423 shows +15% weekday preference while gift items show the opposite. This heterogeneity justifies segment-specific inventory policies, an insight that PDP averaging would mask.

Fig. 18 illustrates an operational dashboard integrating all four levels, providing forecast visualization, SHAP-based key drivers, natural language summary, and recommended inventory action.

VI. DISCUSSION

The results reveal three themes (momentum dominance in feature importance, complementarity across XAI levels, and practical deployment considerations) discussed below alongside limitations and future directions.

A. THEORETICAL IMPLICATIONS

The 28-day rolling mean’s dominance (PFI score 191.23) aligns with demand smoothing theory [54] and the Bullwhip Effect [55]: shorter windows amplify noise and cause procurement overreaction, while longer windows respond too slowly to genuine demand shifts. The 28-day period balances these tensions, roughly matching a monthly business cycle. Ablation results support this interpretation: `rolling_std` features (capturing volatility) rank highly because they signal when the rolling mean may be misleading due to unusual variance.

Temporal features contribute through a contextual mechanism. Day-of-week and month indicators do not predict demand directly; rather, they modulate the interpretation of rolling statistics. A rolling mean of 50 units carries different implications on Monday (office supply restocking) versus Saturday (gift purchasing). The ablation finding that temporal feature removal degraded performance (R^2 decline of 0.012) confirms these contextual signals help the model interpret rolling statistics correctly.

As contextualized in Section V, the R^2 of 0.243 reflects three compounding factors: the intrinsic variability of SKU-level intermittent demand, the deliberate exclusion of external explanatory variables, and the structural

Four-Level Hybrid XAI Framework: From Global to Local Insights

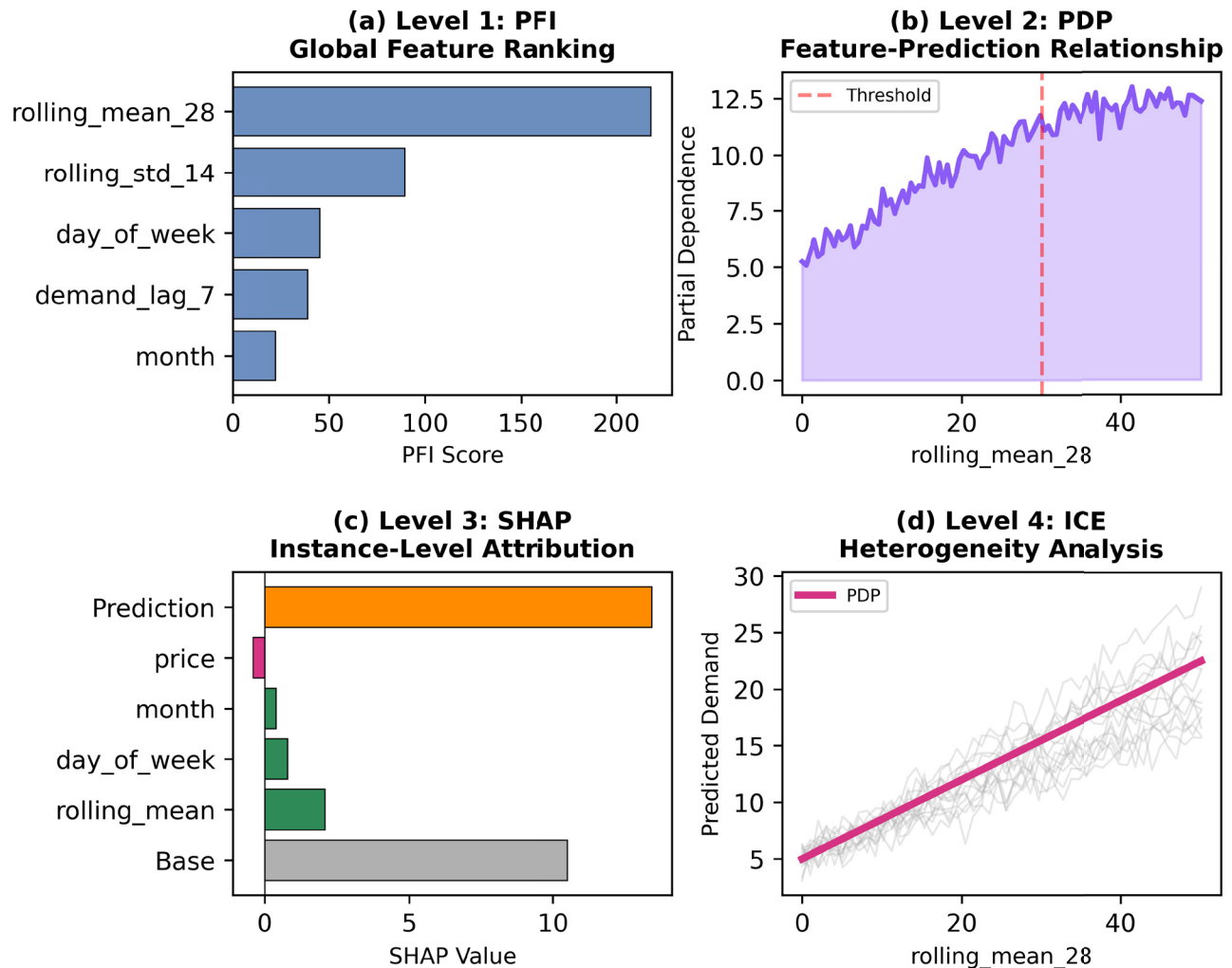


FIGURE 17. Four-level XAI comparison. (a) PFI: global ranking. (b) PDP: feature relationships. (c) SHAP: instance attribution. (d) ICE: heterogeneity detection.

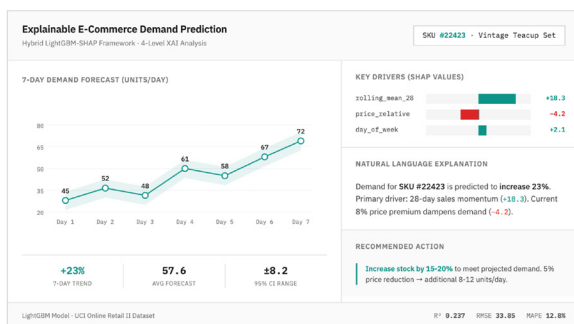


FIGURE 18. Operational XAI dashboard for SKU #22423 integrating forecast, SHAP drivers, natural language explanation, and inventory recommendation.

heterogeneity of the B2B wholesale dataset. Accordingly, the feature relationships and policy recommendations discussed

in this section should be interpreted as directional insights for this dataset rather than definitive prescriptions generalizable to all retail contexts. That said, the modest predictive performance strengthens rather than weakens the case for XAI: SHAP explanations help planners understand why specific forecasts are uncertain, and ICE heterogeneity analysis identifies which products exhibit more predictable patterns worth modeling separately.

The four XAI levels exhibit theoretical complementarity rather than redundancy. PFI and PDP answer different global questions (what matters? how does it relate?), while SHAP and ICE answer different local questions (why this prediction? does the pattern hold for all products?). A planner using only SHAP would miss strategic insights about feature monitoring priorities; using only PDP would miss heterogeneity that justifies segment-specific policies.

B. PRACTICAL IMPLICATIONS

Each XAI level maps to specific operational workflows. PFI rankings inform data infrastructure investment: our top 10 features concentrate 67% of cumulative importance, so data pipelines feeding rolling_mean_28 warrant the highest reliability standards, while features below the top 20 can tolerate looser service-level agreements (SLAs). PDP analysis converts abstract model behavior into concrete inventory rules: when rolling_mean_28 crosses specific thresholds, a category manager can set corresponding reorder triggers.

SHAP enables exception-based forecast review. Rather than validating 4,000 daily SKU forecasts manually, planners flag predictions where SHAP contributions deviate from historical patterns, distinguishing genuine demand shifts from data quality issues. ICE heterogeneity analysis justifies when to abandon uniform policies: parallel curves validate global rules, while crossing curves signal that blanket policies will systematically misallocate inventory across product segments.

XAI outputs also serve ongoing model maintenance. PFI drift over time signals changing demand dynamics; PDP shape changes indicate relationship shifts. Monitoring these patterns enables proactive retraining before forecast accuracy visibly degrades.

C. LIMITATIONS AND FUTURE DIRECTIONS

We acknowledge several limitations that scope interpretation and suggest research directions.

The single-dataset validation (UCI Online Retail II, UK-based wholesaler) limits generalizability claims. Brick-and-mortar retailers face different demand patterns (foot traffic seasonality, local weather effects) than online wholesalers. Grocery categories exhibit perishability constraints absent from giftware. While our framework architecture transfers to these contexts, the specific feature importance rankings and PDP shapes may not. Multi-domain validation across retail formats would strengthen the practical recommendations.

Our model assumes stationary feature-outcome relationships, an assumption retail dynamics routinely violate. The 2020-2022 pandemic period demonstrated how quickly demand patterns can shift: features calibrated on 2019 data failed when work-from-home changed consumption timing. Production deployment requires monitoring protocols to detect relationship drift (via PDP shape comparison over time) and trigger retraining before forecast accuracy visibly degrades. The XAI outputs themselves can serve as drift detectors, a secondary benefit of interpretability.

The feature set excludes external factors that practitioners consider essential: promotional calendars, holiday flags, economic indicators, weather data. These omissions likely cap accuracy below achievable levels. We excluded them deliberately to isolate the contribution of transaction-derived features and XAI methodology, but production systems should integrate external signals. The framework's modular

design accommodates these additions without architectural changes.

The `price_relative` feature divides current unit price by the product's mean price across all available data, which introduces minor information leakage from future observations. Because product prices in this dataset are largely time-invariant, the practical effect on model performance is negligible; however, strictly causal implementations should use expanding-window price averages.

SHAP computation scales poorly to millions of daily predictions. TreeSHAP's $O(TLD^2)$ complexity (T trees, L leaves, D depth) becomes burdensome at scale. Production systems may need sampling strategies (explain 1% of predictions, flag deviations) or approximate methods (KernelSHAP) that trade explanation fidelity for speed. This accuracy-latency tradeoff deserves explicit treatment in deployment planning.

Finally, our one-day-ahead focus reflects the dataset's daily aggregation but limits planning horizons. Multi-step forecasting (7-day, 28-day) would support longer procurement cycles but introduces error accumulation as predictions feed back into lag features. Probabilistic forecasting, outputting confidence intervals rather than point estimates, would better characterize the uncertainty inherent in intermittent demand. These extensions represent natural next steps.

VII. CONCLUSION

We introduced a four-level XAI framework that pairs LightGBM prediction with layered interpretability for retail demand forecasting. The architecture progresses from global feature ranking (PFI) through marginal effect visualization (PDP) to instance-level attribution (SHAP) and heterogeneity detection (ICE). The fourth level, ICE analysis, extends Wang et al.'s [8] global-only approach by revealing when averaged PDP curves mask divergent product behavior, a critical insight for inventory policy differentiation.

Validation on UCI Online Retail II (1.07 million transactions, 4,631 SKUs) confirmed both prediction performance and explanation utility. LightGBM achieved RMSE of 33.71 and R^2 of 0.243 on daily SKU-level demand, outperforming six baselines including Optuna-tuned CatBoost (33.83 RMSE, $p=0.917$), XGBoost (33.89 RMSE, $p < 0.001$), Random Forest, Linear Regression, Moving Average, and Naive Lag-1. All three gradient boosting variants converged within 0.5% RMSE, confirming algorithmic convergence on tabular data. The 20% improvement over naive forecasting (MASE = 0.799) validated practical utility. The modest R^2 reflects three compounding factors: the intrinsic variability of SKU-level intermittent demand, the deliberate exclusion of external explanatory variables, and the structural heterogeneity of the B2B wholesale dataset. Industry benchmarks consider 50–70% accuracy challenging for such lumpy demand patterns, and the findings should be interpreted within these constraints.

Feature analysis and ablation experiments confirmed demand momentum as the dominant driver: the 28-day rolling

mean's PFI score (191.23) exceeded all others, rolling feature removal caused the largest degradation ($\Delta R^2 = -0.064$), and rolling statistics alone explained 76% of the full model's variance ($R^2 = 0.184$).

The framework demonstrated that accuracy and interpretability need not trade off. PFI guides data infrastructure investment. PDP shapes inform reorder threshold rules. SHAP decomposition enables exception-based forecast review. ICE heterogeneity analysis, the key contribution, provides quantified indices (IVI, ICI) that transform visual curve inspection into actionable metrics for automated policy differentiation. A proof-of-concept inventory simulation demonstrated that ICE-informed segmentation produces measurably different service-level/holding-cost tradeoffs compared to uniform policies, though aggregate performance improvement requires production-grade optimization with realistic cost models and per-segment service targets.

Future work will extend to multi-step forecasting with error propagation analysis and incorporate external signals such as promotions, holidays, and weather. Additionally, we plan to validate ICE-based segmentation with realistic holding/stockout cost models across additional retail domains.

ACKNOWLEDGMENT

The authors thank the UCI Machine Learning Repository for providing the Online Retail II dataset. They acknowledge the open-source communities behind LightGBM, SHAP, scikit-learn, and Python.

DATA AVAILABILITY

The UCI Online Retail II dataset is publicly available at the UCI Machine Learning Repository [47]. Code for reproducing experiments will be made available at publication time.

REFERENCES

- [1] J. R. Stock and D. M. Lambert, *Strategic Logistics Management*. New York, NY, USA: McGraw-Hill, 2001.
- [2] S. Makridakis, E. Spiliotis, and V. Assimakopoulos, "Statistical and machine learning forecasting methods: Concerns and ways forward," *PLO ONE*, vol. 13, no. 3, Mar. 2018, Art. no. e0194889.
- [3] S. Makridakis, E. Spiliotis, and V. Assimakopoulos, "M5 accuracy competition: Results, findings, and conclusions," *Int. J. Forecasting*, vol. 38, no. 4, pp. 1346–1364, Oct. 2022.
- [4] V. Chamola, V. Hassija, A. R. Sulthana, D. Ghosh, D. Dhingra, and B. Sikdar, "A review of trustworthy and explainable artificial intelligence (XAI)," *IEEE Access*, vol. 11, pp. 78994–79015, 2023.
- [5] A. Agrawal, J. Gans, and A. Goldfarb, "Economic policy for artificial intelligence," *Innov. Policy Economy*, vol. 19, no. 1, pp. 139–159, 2019.
- [6] E. Brynjolfsson, D. Rock, and C. Syverson, "The productivity J-curve: How intangibles complement general purpose technologies," *Amer. Econ. J. Macroeconomics*, vol. 13, no. 1, pp. 333–372, Jan. 2021.
- [7] A. Adadi and M. Berrada, "Peeking inside the black-box: A survey on explainable artificial intelligence (XAI)," *IEEE Access*, vol. 6, pp. 52138–52160, 2018.
- [8] J. Wang, Y. Wang, and I. Othman, "Explainable E-commerce transaction prediction using LightGBM and permutation importance," *IEEE Access*, vol. 14, pp. 10153–10169, 2026.
- [9] G. E. P. Box, G. M. Jenkins, G. C. Reinsel, and G. M. Ljung, *Time Series Analysis: Forecasting and Control*. Hoboken, NJ, USA: Wiley, 2015.
- [10] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*. Melbourne, VIC, Australia: OTexts, 2021.
- [11] G. E. P. Box and G. M. Jenkins, "Some recent advances in forecasting and control," *J. Roy. Stat. Soc. Ser. C*, vol. 17, no. 2, pp. 91–109, 1968.
- [12] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001.
- [13] V. N. Vapnik, *The Nature of Statistical Learning Theory*. New York, NY, USA: Springer, 2000.
- [14] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, Nov. 1997.
- [15] Y. Bengio, P. Simard, and P. Frasconi, "Learning long-term dependencies with gradient descent is difficult," *IEEE Trans. Neural Netw.*, vol. 5, no. 2, pp. 157–166, Mar. 1994.
- [16] B. Lim, S. Ö. Arik, N. Loeff, and T. Pfister, "Temporal fusion transformers for interpretable multi-horizon time series forecasting," *Int. J. Forecasting*, vol. 37, no. 4, pp. 1748–1764, Oct. 2021.
- [17] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 5998–6008.
- [18] S. J. Taylor and B. Letham, "Forecasting at scale," *Amer. Statistician*, vol. 72, no. 1, pp. 37–45, 2018.
- [19] B. N. Oreshkin, D. Carпов, N. Chapados, and Y. Bengio, "N-BEATS: Neural basis expansion analysis for interpretable time series forecasting," in *Proc. Int. Conf. Learn. Represent.*, 2020.
- [20] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *Ann. Statist.*, vol. 29, no. 5, pp. 1189–1232, Oct. 2001.
- [21] R. Shwartz-Ziv and A. Armon, "Tabular data: Deep learning is not all you need," *Inf. Fusion*, vol. 81, pp. 84–90, May 2022.
- [22] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Aug. 2016, pp. 785–794.
- [23] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 3146–3154.
- [24] C. Bentéjac, A. Csörgő, and G. Martínez-Muñoz, "A comparative analysis of gradient boosting algorithms," *Artif. Intell. Rev.*, vol. 54, no. 3, pp. 1937–1967, Mar. 2021.
- [25] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin, "CatBoost: Unbiased boosting with categorical features," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 6639–6649.
- [26] A. Fisher, C. Rudin, and F. Dominici, "All models are wrong, but many are useful: Learning a variable's importance by studying an entire class of prediction models simultaneously," *J. Mach. Learn. Res.*, vol. 20, no. 177, pp. 1–81, 2019.
- [27] C. Molnar, *Interpretable Machine Learning: A Guide for Making Black Box Models Explainable*. Victoria, BC, Canada: Leanpub, 2022.
- [28] D. W. Apley and J. Zhu, "Visualizing the effects of predictor variables in black box supervised learning models," *J. Roy. Stat. Soc. Ser. B, Stat. Methodol.*, vol. 82, no. 4, pp. 1059–1086, Sep. 2020.
- [29] S. Lundberg and S. Lee, "A unified approach to interpreting model predictions," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 4765–4774.
- [30] S. M. Lundberg, G. Erion, H. Chen, A. DeGrave, J. M. Prutkin, B. Nair, R. Katz, J. Himmelfarb, N. Bansal, and S.-I. Lee, "From local explanations to global understanding with explainable AI for trees," *Nature Mach. Intell.*, vol. 2, no. 1, pp. 56–67, Jan. 2020.
- [31] A. Goldstein, A. Kapelner, J. Bleich, and E. Pitkin, "Peeking inside the black box: Visualizing statistical learning with plots of individual conditional expectation," *J. Comput. Graph. Statist.*, vol. 24, no. 1, pp. 44–65, Jan. 2015.
- [32] M. T. Ribeiro, S. Singh, and C. Guestrin, "Why should i trust you?: Explaining the predictions of any classifier," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2016, pp. 1135–1144.
- [33] S. Wachter, B. Mittelstadt, and C. Russell, "Counterfactual explanations without opening the black box: Automated decisions and the GDPR," *Harvard J. Law Technol.*, vol. 31, no. 2, pp. 841–887, 2018.
- [34] S. Jain and B. C. Wallace, "Attention is not explanation," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics*, 2019, pp. 3543–3556.
- [35] A. De Caigny, K. Coussement, and K. W. De Bock, "A new hybrid classification algorithm for customer churn prediction based on logistic regression and decision trees," *Eur. J. Oper. Res.*, vol. 269, no. 2, pp. 760–772, Sep. 2018.
- [36] L. N. Mishra and B. Senapati, "FraudFusion: A multi-modal ensemble framework for anomaly-based fraud detection in retail POS systems," *IEEE Access*, vol. 14, pp. 10436–10456, 2026.

- [37] W. Hilal, S. A. Gadsden, and J. Yawney, "Financial fraud: A review of anomaly detection techniques and recent advances," *Expert Syst. Appl.*, vol. 193, May 2022, Art. no. 116429.
- [38] K. J. Ferreira, B. H. A. Lee, and D. Simchi-Levi, "Analytics for an online retailer: Demand forecasting and price optimization," *Manuf. Service Oper. Manage.*, vol. 18, no. 1, pp. 69–88, Feb. 2016.
- [39] Y. Zhang and X. Chen, "Explainable recommendation: A survey and new perspectives," *Found. Trends Inf. Retr.*, vol. 14, no. 1, pp. 1–101, Mar. 2020.
- [40] F. Petropoulos, "Forecasting: Theory and practice," *Int. J. Forecasting*, pp. 845–1168, 2022.
- [41] M. A. Jahin, A. Shahriar, and M. A. Amin, "MCDNF: Supply chain demand forecasting via an explainable multi-channel data fusion network model," *Evol. Intell.*, vol. 18, no. 3, Jun. 2025, Art. no. 66.
- [42] S. Alam, M. Abdul Rahman, and B. Sadeqa, "DriftGuard: A hierarchical framework for concept drift detection and remediation in supply chain forecasting," 2026, *arXiv:2601.08928*.
- [43] S. K. Jauhar, S. Harinath, V. Krishnaswamy, and S. K. Paul, "Explainable artificial intelligence to improve the resilience of perishable product supply chains by leveraging customer characteristics," *Ann. Oper. Res.*, vol. 354, no. 1, pp. 103–142, Nov. 2025.
- [44] M. Nasser, T. Falatouri, P. Brandtner, and F. Darbanian, "Applying machine learning in retail demand prediction—A comparison of tree-based ensembles and long short-term memory-based deep learning," *Appl. Sci.*, vol. 13, no. 19, p. 11112, Oct. 2023.
- [45] L. N. Mishra and B. Senapati, "Retail resilience engine: An agentic AI framework for building reliable retail systems with test-driven development approach," *IEEE Access*, vol. 13, pp. 50226–50243, 2025.
- [46] C. Bergmeir and J. M. Benítez, "On the use of cross-validation for time series predictor evaluation," *Inf. Sci.*, vol. 191, pp. 192–213, May 2012.
- [47] D. Chen, "Online retail II dataset," UCI Mach. Learn. Repository, Univ. California, Irvine, Irvine, CA, USA, Tech. Rep., 2019, doi: [10.24432/C5CG6D](https://doi.org/10.24432/C5CG6D).
- [48] R. J. Hyndman and A. B. Koehler, "Another look at measures of forecast accuracy," *Int. J. Forecasting*, vol. 22, no. 4, pp. 679–688, Oct. 2006.
- [49] A. A. Syntetos, J. E. Boylan, and J. D. Croston, "On the categorization of demand patterns," *J. Oper. Res. Soc.*, vol. 56, no. 5, pp. 495–503, May 2005.
- [50] T. R. Willemain, C. N. Smart, and H. F. Schwarz, "A new approach to forecasting intermittent demand for service parts inventories," *Int. J. Forecasting*, vol. 20, no. 3, pp. 375–387, Jul. 2004.
- [51] A. A. Syntetos and J. E. Boylan, "On the bias of intermittent demand estimates," *Int. J. Prod. Econ.*, vol. 71, nos. 1–3, pp. 457–466, May 2001.
- [52] L. Grinsztajn, E. Oyallon, and G. Varoquaux, "Why do tree-based models still outperform deep learning on typical tabular data?" in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 507–520.
- [53] F. X. Diebold and R. S. Mariano, "Comparing predictive accuracy," *J. Bus. Econ. Statist.*, vol. 13, no. 3, pp. 253–263, 1995.
- [54] H. L. Lee, V. Padmanabhan, and S. Whang, "Information distortion in a supply chain: The bullwhip effect," *Manage. Sci.*, vol. 43, no. 4, pp. 546–558, 1997.
- [55] F. Chen, Z. Drezner, J. K. Ryan, and D. Simchi-Levi, "Quantifying the bullwhip effect in a simple supply chain: The impact of forecasting, lead times, and information," *Manage. Sci.*, vol. 46, no. 3, pp. 436–443, Mar. 2000.



LALIT NARAYAN MISHRA (Senior Member, IEEE) received the M.S. degree in computer science from Manipal University, India, in 2013. He is currently a Senior Manager of software engineering with Lowe's Companies Inc., Mooresville, NC, USA, where he leads three cross-functional teams building AI-driven retail platforms. He is the author of *Spring*, *Hibernate*, *Data Modeling*, *REST*, and *TDD* (Amazon), in 2015, and has published in IEEE Access, including the Retail

Resilience Engine Framework, in 2025, and the Fraud Fusion ensemble fraud-detection system, in 2026. He serves as a Peer Reviewer for *PLOS ONE* and *IGI Global journals*. He has also served as a judge and mentor at multiple university hackathons. His research interests include explainable artificial intelligence, retail demand forecasting, ensemble machine learning, and agentic AI systems.



BISWARANJAN SENAPATI (Senior Member, IEEE) is a Platinum-Level ERP-SAP Consultant with more than 22 years of industry experience in domain consulting, business consulting, and solution architecture for large-scale ERP system design, development, implementation, roll-out, support, and business process re-engineering capacity in the supply chain, chemicals, hi-tech, CPG, pharmaceuticals, wholesale, fashion retail, health care, and manufacturing domains. He has previously worked at Parker Hannifin, Home Depot, SONY, Gordon Foods, Constellation Brands, Olin Corporations, HPE, Ecolabs, TSC, Hanes Brands, PVH, Applexus, ECCO Denmark, Mindtree Germany, Ciber USA, Cognizant, Capgemini, and Deloitte Consulting, and the SAP Group of Consulting Companies.



SAROJ KUMAR NAYAK (Member, IEEE) received the B.Sc. (Hons.) and M.Sc. degrees in mathematics, and the M.Tech. degree in computer science from Utkal University. He is currently an ERP Leader with more than 20 years of experience delivering Oracle-based solutions for Fortune 100 enterprises. He pairs deep expertise in Oracle E-Business Suite (EBS) with specialization in AI/ML and generative AI to provide consulting, solution architecture, ERP implementation, business process analysis, and process re-engineering. He has led large and mid-size programs across supply chain, manufacturing, retail, financial services, and healthcare. Currently at Cognizant Technology Solutions, USA, he focuses on healthcare solutioning across platforms, leveraging Azure Cloud, Kafka, Cayman, AI/ML for imaging, prompt engineering, and Oracle technologies, including SOA, SQL, Oracle Forms, Workflow, and data warehousing. His technical toolkit spans Python, R, big data (Hadoop), Tableau, Generative AI and ML models, PL/SQL, SOA, Agile methodologies, and cross-industry business process mapping. He has completed certificate programs in AI/ML and generative AI from Purdue University and a Data Scientist Program from Simplilearn, and an advanced course on the RAISE Specification Language and Duration Calculus at the International Institute for Software Technology, United Nations University, Macau. He has received multiple industry accolades and awards.



AMIT J. RANGARI (Senior Member, IEEE) is currently a Lead Software Engineer in DevOps and performance with JPMorgan Chase & Co., Atlanta, GA, USA. A Published Researcher in software engineering, his work focuses on CI/CD automation, platform resilience, containerization, and cloud-native architectures. With more than 20 years of experience across the banking, telecommunications, and pharmaceutical sectors, he has led large-scale DevOps transformations for Fortune 500 clients. He is a certified AWS Cloud Practitioner and a peer reviewer for IEEE-affiliated journals in artificial intelligence, healthcare systems, and precision agriculture. He combines his technical expertise with a commitment to advancing industry practices, regularly contributing to academic discourse through publications and presentations.

...